UNIVERSIDADE LaSalle
Viver é evoluir.

NOVA GRADUAÇÃO UNIVERSIDADE LA SALLE

UNIDADE 2

Requisitos de SOFTWARE



INOVAÇÃO E TECNOLOGIA

Especificação e Registro de Requisitos Funcionais

Prezado estudante,

Estamos começando uma unidade desta disciplina. Os textos que a compõem foram organizados com cuidado e atenção, para que você tenha contato com um conteúdo completo e atualizado tanto quanto possível. Leia com dedicação, realize as atividades e tire suas dúvidas com os tutores. Dessa forma, você, com certeza, alcançará os objetivos propostos para essa disciplina.

Objetivo Geral

Conhecer as formas de especificar e registrar requisitos funcionais de software.

Editora
LaSalle

Parte 1

Diagrama de Casos de Uso

O conteúdo deste livro
é disponibilizado
por SAGAH.

SAGAH
SOLUÇÕES
EDUCACIONAIS
INTEGRADAS

Diagrama de casos de uso

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Identificar os atores que interagem com um sistema de *software*.
- Projetar casos de uso do sistema.
- Representar os relacionamentos no diagrama de casos de uso.

Introdução

Ao darmos início ao desenvolvimento de um *software*, a primeira questão que surge é "O que deve ser construído?". Embora a pergunta seja simples, a resposta não é. Elicitar requisitos é uma tarefa complexa, que exige tempo, planejamento e experiência. Dependendo do contexto, da complexidade do projeto e dos interesses conflitantes, identificar os requisitos sob o ponto de vista dos diversos *stakeholders* pode ser um desafio até mesmo para os analistas de requisitos mais experientes.

Uma das ferramentas que podem ser utilizadas para apoiar esse processo e representar graficamente os requisitos funcionais é o diagrama de casos de uso, que oferece uma forma simples de comunicação com os *stakeholders* em torno das funcionalidades e dos serviços que serão oferecidos aos usuários. O diagrama de casos de uso pode ser usado desde as etapas iniciais da elicitação de requisitos, como um instrumento de apoio para as entrevistas, reuniões ou *workshops*. Ele pode também apoiar o gerente de projetos como uma forma de documentação gráfica do escopo funcional.

Neste capítulo você vai estudar a elaboração de um diagrama de casos de uso, iniciando pela identificação dos atores que interagem com o *software*. Em seguida vai ver como projetar os casos de uso em si e como identificar e representar os relacionamentos dos atores com outros atores, dos atores com os casos de uso e dos casos de uso entre si.

1 Identificação dos atores

O **diagrama de casos de uso** faz parte do conjunto de diagramas de comportamento propostos pela UML (*Unified Modeling Language*), conforme ilustra a Figura 1.

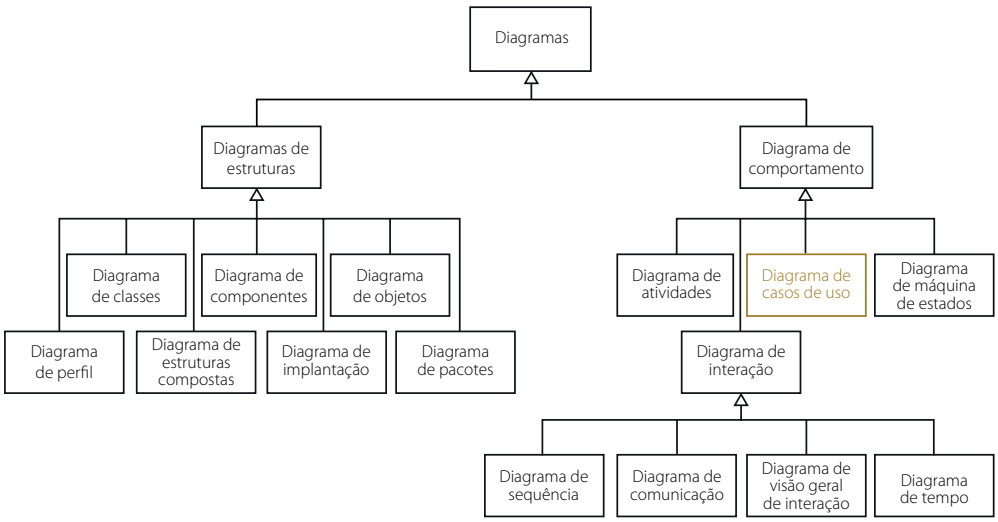


Figura 1. Diagramas da UML.

Fonte: Adaptada de Object Management Group (2017).

A primeira proposta de utilização de uma abordagem centrada no usuário com base em casos de uso foi feita por Ivar Jacobson, em 1992, que, na época, trabalhava na Erickson. Junto com James Rumbaugh e Grady Booch, Jacobson criou a linguagem de modelagem UML. Essa foi a base para outras abordagens centradas no usuário, incluindo as abordagens ágeis.

O diagrama de casos de uso representa o usuário e suas interações com o sistema, portanto ele é uma ferramenta útil quando o sistema em desenvolvimento é um sistema de informação, ou seja, quando existem diversas interações desse tipo para serem projetadas. Sistemas cuja complexidade não esteja na interação com o usuário, mas sim em seu processamento interno, não se beneficiarão desse tipo de representação (DENNIS; WIXON; ROTH, 2012).

Não existe um primeiro passo obrigatório para iniciar a construção do diagrama de casos de uso. Pode-se dar início tanto pela identificação dos próprios casos de uso quanto pela identificação dos atores que irão interagir com o *software.*

De acordo com a UML (OBJECT MANAGEMENT GROUP, 2017, documento *on-line*), "[...] um ator especifica um papel que é desempenhado por um usuário ou qualquer outro sistema que interage com o sujeito". Entende-se por **sujeito** o sistema que está sendo construído. Um **ator** reside fora das fronteiras da aplicação, portanto ele pode ser compreendido como um papel que um usuário pode assumir ao interagir com o *software* para executar uma função. É uma espécie de chapéu que ele pode usar naquele momento da interação.

O ator pode ainda ser um *hardware* especial ou outro *software* com o qual o sistema interage e que está fora das fronteiras da aplicação. Mencionamos aqui *hardware* especial como forma de destacar que só devem ser representados os equipamentos que desempenham uma função específica e de interesse do produto. Por exemplo, não é necessário representar a impressora por meio da qual será impresso um relatório, mas pode ser de interesse representar determinado sensor que transmite dados que são interpretados pelo *software* e que desencadeiam ações.



Saiba mais

Raciocínio similar deve ser feito para os demais *softwares* com os quais o sistema interage. Não é necessário representar como ator o banco de dados, por exemplo, mas pode ser importante representar o sistema da operadora de cartão de crédito se o caso de uso de pagamento de uma transação precisa se comunicar com a operadora diretamente.

Como representar os atores?

Em um diagrama de casos de uso, um ator é representado por uma figura humana simplificada, que é desenhada na forma de um boneco palito, ou *stickman,* tendo abaixo dele o nome do papel que desempenha, conforme ilustra a Figura 2.

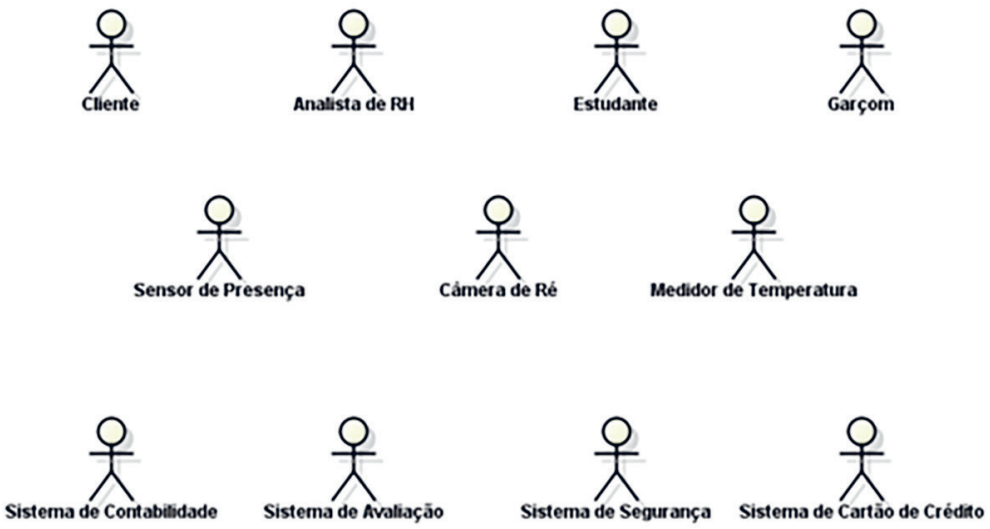


Figura 2. Representação dos atores em diagramas de casos de uso.

Na primeira fileira de atores da Figura 2 podemos ver exemplos de atores humanos, como Cliente, Analista de RH, Estudante e Garçom. Já na segunda fileira temos exemplos de atores que representam *hardwares* especiais, como Sensor de Presença, Câmera de Ré e Medidor de Temperatura. Na última fileira temos exemplos de atores que representam *softwares* com os quais o sistema em desenvolvimento deverá interagir*:* Sistema de Avaliação, Sistema de Segurança e Sistema de Cartão de Crédito.

Todo ator deve ter, obrigatoriamente, um nome associado a ele. Não é recomendado utilizar a denominação "Usuário", pois é muito genérica. Recomenda-se a utilização de um nome mais significativo e específico para o ator, como nos exemplos da Figura 2. Quando se tratar de um *hardware* ou outro *software*, pode-se também usar um estereótipo, indicando que se trata de um sistema. Isto é feito com a ***tag*** `<<system>>` e seu uso vai depender de como a ferramenta de modelagem UML faz essa implementação.

De acordo com a UML (OBJECT MANAGEMENT GROUP, 2017), existem ainda outras duas formas alternativas de representar os atores, mas elas praticamente não são utilizadas: o **retângulo** (como uma classe da orientação a objetos) e o **ícone** (uma figura). As ferramentas de modelagem atualmente no mercado geralmente implementam o ator como um *stickman*.

Um ator pode ser **primário** (quando dá início à ação) ou **secundário** (quando participa da ação ou apenas recebe os resultados da ação). Não existe diferença na sua representação, mas é comum que os atores primários fiquem do lado esquerdo do diagrama de casos de uso, pois é como lemos as coisas no mundo ocidental: da esquerda para a direita e de cima para baixo. Portanto, não é uma regra, mas sim uma boa prática colocá-los do lado esquerdo.

Como identificar os atores?

Para identificar os atores de um sistema é importante iniciar identificando os usuários humanos que irão utilizar as funcionalidades. Em seguida, deve-se identificar com que dispositivos de *hardware* ou outros sistemas de *software* ele irá interagir. Algumas perguntas podem ajudar, acompanhe (GUEDES, 2018):

- Que tipos de usuários poderão utilizar o sistema?
- Quais usuários estão interessados ou utilizarão quais funcionalidades e serviços do *software*?
- Quem fornecerá informações ao sistema?
- Quem utilizará as informações do sistema?
- Quem poderá alterar ou mesmo excluir informações do sistema?
- Existe algum outro *software* que interagirá com o sistema?
- Existe algum *hardware* especial (como um robô, por exemplo) que interagirá com o *software*?

Essas questões podem ser complementadas com as que foram propostas por Wiegers e Beatty (2013), listadas a seguir:

- Quem é notificado quando alguma coisa ocorre no sistema?
- Quem provê informações ou serviços para o sistema?
- Quem o sistema ajuda a responder ou a completar uma tarefa?

Seidl *et al.* (2012) destacam ainda a importância de questionar o seguinte:

- Quem é responsável pela administração do sistema?

O primeiro contato com o patrocinador do projeto apontará para os *stakeholders* que servirão de fonte de informação para a elicitação de requisitos. Alguns desses *stakeholders* serão também usuários do sistema posteriormente

e terão um ator para os representar na especificação. Mas tenha em mente que um ator geralmente representa uma classe de usuários e que nem todos os *stakeholders* que serão fonte de informação serão usuários ou terão seus papéis associados a atores.



Exemplo

Um especialista em tributação pode ser uma fonte de informação para esclarecer detalhes tributários, mas pode não ser um usuário do sistema.

Desde o primeiro contato com o patrocinador, o analista de requisitos deve tentar identificar as classes de usuários (atores).

2 Projeto dos casos de uso

Um caso de uso é uma execução completa de uma funcionalidade que entrega um valor para o usuário que a utiliza. Esse conceito é muito importante para que tenhamos em mente o que realmente significa um caso de uso — ele é uma funcionalidade completa e que entrega algum valor. É comum vermos diagramas de caso de uso com diversos casos de uso que não são casos de uso em si, mas sim passos de um caso de uso.



Fique atento

Fique atento ao conceito de caso de uso! Quando estiver projetando um diagrama de casos de uso e perceber que, para completar uma funcionalidade, precisa de diversos casos de uso, procure identificar se você não está transformando indevidamente uma etapa do caso de uso em outro caso de uso.

Outra situação é quando determinada etapa se repete em diversos casos de uso e ela pode ser agrupada como um caso de uso que é chamado a partir dos demais em uma relação de inclusão (*include*).



Exemplo

Na dúvida, lembre-se do exemplo do caixa eletrônico. O cliente não vai ao caixa eletrônico apenas para inserir seu cartão e ir embora, ele vai para realizar uma transação, que pode ser um saque, por exemplo. Portanto, Inserir Cartão não é um caso de uso, mas sim um passo de Retirar Dinheiro. Todos os passos necessários para que o cliente possa cumprir o seu objetivo de sacar dinheiro constituem o caso de uso Retirar Dinheiro. Isso inclui identificar-se (qualquer que seja a forma exigida pela tecnologia: senha, biometria etc.), preencher as informações necessárias à operação (valor, tipo de cédula etc.) e concluir a solicitação (sair com o dinheiro).

Já a parte de identificação do cliente pode ser agrupada em um caso de uso Identificar-se, que será chamado a partir de todos os demais casos de uso que precisem que a identificação do usuário seja validada.

Como identificar os casos de uso?

Os casos de uso são funcionalidades que o sistema deve prover, portanto, estamos falando de requisitos funcionais. Isso não quer dizer que cada requisito funcional se transforme em um caso de uso. Não é um relacionamento um para um. Um requisito pode estar em um nível de abstração mais alto e pode se transformar em diversos casos de uso. Por outro lado, um requisito pode estar definido em uma granularidade bem pequena e não constituir um caso de uso em si, mas parte de um caso de uso. Nessa situação, um grupo de requisitos funcionais pode gerar um único caso de uso.

A identificação de casos de uso pode ser feita utilizando-se as diversas técnicas de elicitação de requisitos, como entrevista, observação, reunião, *brainstorming*, grupo focal, sessão JAD (Joint Application Design) etc. O importante é que todo o escopo do projeto ou da versão/*sprint* tenha sido coberto.

Podemos utilizar estas perguntas, que podem ajudar na identificação dos casos de uso (SEIDL *et al.*, 2012):

- Quais são as tarefas principais que um ator deverá executar?
- Um ator deseja consultar ou até mesmo modificar alguma informação que existe no sistema?
- Algum ator precisa informar o sistema sobre mudanças que ocorreram em outros sistemas?
- Algum ator precisa ser informado sobre eventos inesperados dentro do sistema?

Como representar os casos de uso?

Um caso de uso é representado por uma elipse com seu nome dentro ou logo abaixo. O caso de uso representa uma ação e, portanto, deve ser nomeado com um verbo no infinitivo, conforme ilustra a Figura 3. Ele faz parte do sistema que está sendo definido e, portanto, fica dentro do retângulo que representa o sistema, ao contrário dos atores, que ficam do lado de fora.

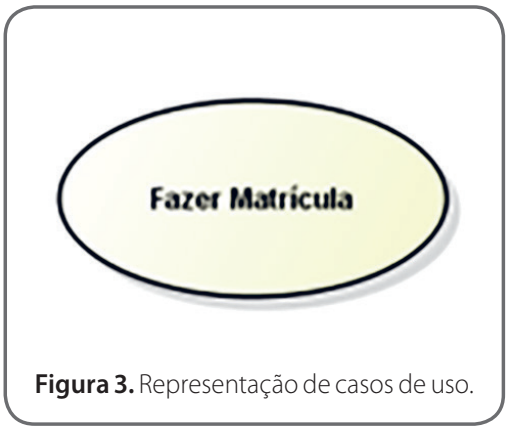


Figura 3. Representação de casos de uso.

Quando o sistema tiver muitas funcionalidades, para que o diagrama de caso de uso não fique excessivamente poluído, pode-se optar por representar apenas os casos de uso que sejam mais importantes e que representem a razão de existir do sistema, os chamados **casos de uso primários**. Podem ser considerados casos de uso secundários os que apenas mantêm cadastros ou os que emitem relatórios/consultas simples. Estes podem ficar fora do diagrama de casos de uso quando ele for muito complexo.

Existe uma situação especial que divide a opinião dos autores relacionada às funcionalidades que são disparadas pelo próprio sistema, por meio de eventos de controle ou pelo atingimento de determinado momento no tempo — por exemplo, quando um processamento é executado diariamente de forma automática, digamos, a partir das 22h. Como representar esse caso específico no diagrama de casos de uso?

Uma forma é considerar que o próprio sistema é um ator e que ele é o responsável por iniciar as funcionalidades que devem ser disparadas por ele sob determinadas circunstâncias. Embora seja uma representação clara e compreensível, o conceito de ator é violado, pois um ator é algo externo ao sistema. Outra forma de realizar essa representação é criar um ator *scheduler*, que pode ser considerado como um "agendador" de tarefas automáticas, ao qual ficam associados os casos de uso dessa natureza.

Apenas o diagrama de casos de uso não é suficiente para expressar toda a complexidade de uma funcionalidade. É necessário que seja elaborada uma especificação de casos de uso, que vai detalhar os passos e condições de execução dos casos de uso. Essa especificação não é objeto deste capítulo e, portanto, não será tratada aqui.

3 Identificação dos relacionamentos

Os relacionamentos no diagrama de casos de uso são chamados de **associações**. Elas podem representar relacionamentos entre os atores e os casos de uso, entre casos de uso entre si e também entre atores entre si. Vamos ver como identificar e representar cada uma delas.

Associação entre atores e casos de uso

Uma associação entre um ator e um caso de uso indica que o ator inicia o caso de uso (geralmente chamado de ator Principal ou ator Primário) ou que o ator recebe os resultados do caso de uso que foi iniciado por outro ator. Essa associação é representada por uma linha contínua que liga o ator ao caso de uso, significando que a informação flui nos dois sentidos. Ela também pode conter uma seta em uma das pontas para indicar o sentido do fluxo da informação ou para indicar quem inicia a comunicação.

É possível que um caso de uso tenha mais do que um ator associado. Isso ocorre quando mais do que um ator é necessário para que o caso de uso seja realizado.

A Figura 4 apresenta um exemplo parcial para um *software* de Solicitação de Transporte. O ator Passageiro pode Solicitar Transporte, Alterar Percurso e Cancelar Corrida. A informação flui nos dois sentidos. O retângulo representa as fronteiras do Sistema de Solicitação de Transporte.

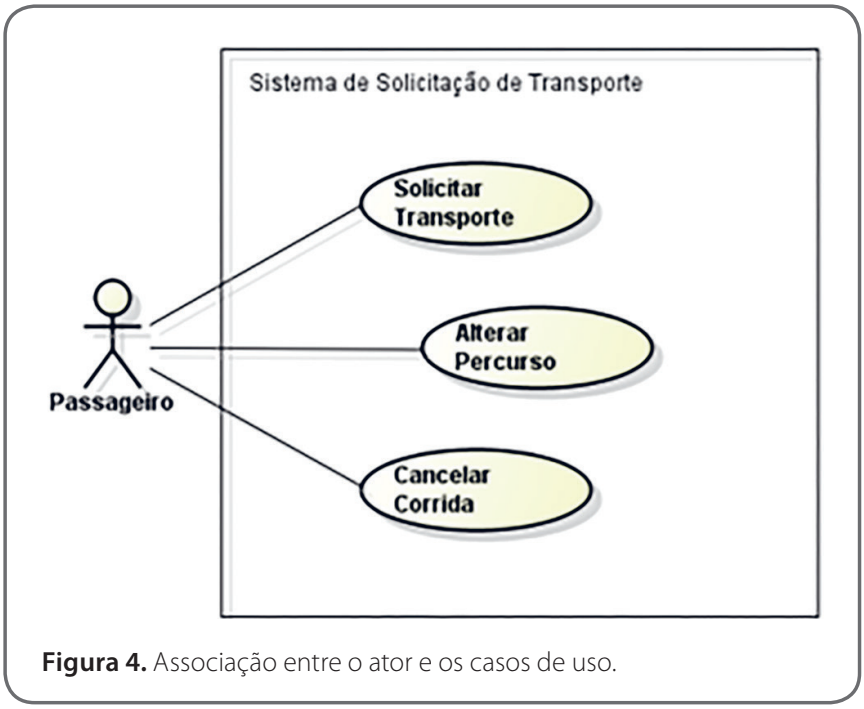


Figura 4. Associação entre o ator e os casos de uso.

Associação entre casos de uso

A associação entre casos de uso pode tomar três formatos: inclusão (*include*), extensão (*extend*) e generalização (ou herança). Optamos, neste capítulo, pela manutenção do termo em inglês, pois é o mais usual, inclusive nas ferramentas que apoiam a modelagem UML.

Relacionamento do tipo *include*

O relacionamento do tipo *include* é utilizado quando o comportamento de um caso de uso está sendo incluído dentro de outro caso de uso. Ele pressupõe que toda vez que o caso de uso que inclui for executado, o caso de uso incluído também será. O caso de uso incluído será executado de uma vez só, quando chamado pelo caso de uso que inclui. Após a sua execução, a execução dos próximos passos do caso de uso que inclui é retomada.



Saiba mais

Podemos entender que o relacionamento do tipo *include* foca no reuso, ou seja, na não repetição de uma mesma especificação em vários lugares diferentes, sendo similar à chamada de uma sub-rotina ou de uma função, muito comum em diversas linguagens de programação.

O *include* é representado por uma seta pontilhada que parte do caso de uso que inclui para o caso de uso que é incluído. É comum que seja também adicionado um estereótipo com a palavra `include` entre dois sinais de menor (<<) e maior (>>), o que normalmente é feito pelas ferramentas de modelagem UML de forma automática.

Vamos acompanhar o exemplo da Figura 5, que reflete uma das funcionalidades de um *site* de compras *on-line*.

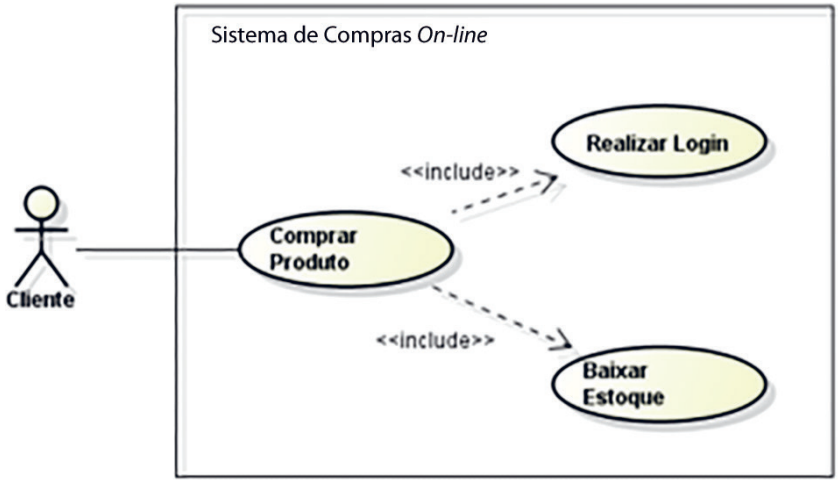


Figura 5. Relacionamento do tipo *include*.

O diagrama de casos de uso representado na Figura 5 diz que o ator Cliente só poderá comprar um produto se estiver logado. Note que a seta pontilhada que une os dois casos de uso aponta do caso de uso que inclui (Comprar Produto), para o caso de uso que é incluído (Realizar Login). Da mesma forma, quando o Cliente realiza a compra, será realizada a baixa no estoque (Baixar Estoque), representada pela relação *include* entre os dois casos de uso.

De acordo com as definições da UMLv2.5.1 (OBJECT MANAGEMENT GROUP, 2017, documento *on-line*),

> O relacionamento de *include* é para ser usado quando existem partes comuns do comportamento de um ou mais casos de uso. Esta parte comum é então extraída em um caso de uso separado, para ser incluída por todos os casos de uso base que tenham esta parte comum. Como o uso principal do relacionamento de *include* é para o reuso de partes comuns, o que sobra em um caso de uso base é normalmente incompleto em si, mas dependente da parte incluída para fazer sentido. Isto é refletido na direção do relacionamento, indicando que o caso de uso base depende da adição e não vice-versa.

Relacionamento do tipo *extend*

O relacionamento do tipo *extend* é utilizado quando o comportamento de um caso de uso estende o comportamento de outro caso de uso, sob circunstâncias específicas. O relacionamento *extend* especifica como e quando o comportamento definido no caso de uso que estende pode ser inserido dentro do comportamento definido do caso de uso estendido. Essa extensão pode ocorrer em um ou mais pontos de extensão do caso de uso estendido. O *extend* é usado quando o comportamento é adicionado de forma condicional, ou seja, sob certas circunstâncias.

O *extend* é representado por seta pontilhada que parte do caso de uso que estende para o caso de uso que é estendido. É comum que seja também adicionado um estereótipo com a palavra `extend` entre dois sinais de menor (<<) e maior (>>), o que normalmente é feito pelas ferramentas de modelagem UML de forma automática.

Vamos acompanhar o exemplo da Figura 6, que representa as formas de pagamento que o ator Cliente tem ao realizar uma compra *on-line*. O Cliente, ao pagar a compra, poderá optar pelo pagamento na forma de boleto bancário ou por meio do cartão de crédito. O retângulo externo representa as fronteiras do Sistema de Compras *On-line*.

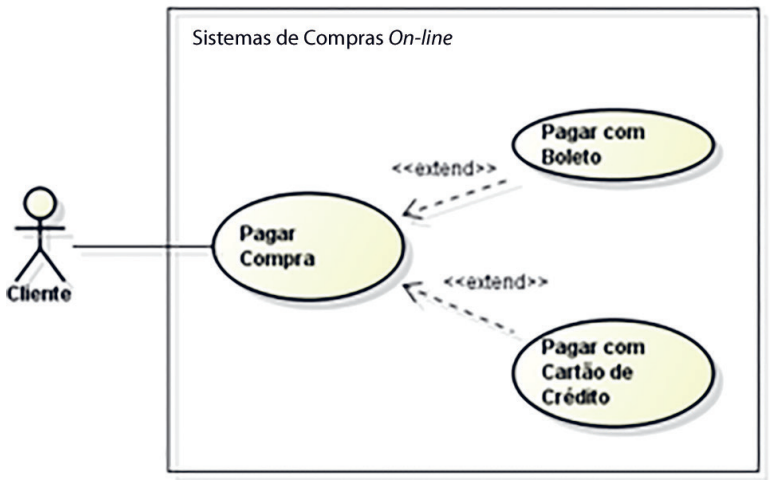


Figura 6. Relacionamento do tipo *extend*.

É possível ainda que seja identificado explicitamente no diagrama quais são os pontos de extensão, conforme ilustrado na Figura 7. Nesse caso, essa identificação fica dentro da elipse que representa o caso de uso, com a expressão *extension points* (pontos de extensão) logo abaixo do nome do caso de uso, em um compartimento separado por uma linha contínua.

Isso significa que, se essa condição for satisfeita, o caso de uso que estende será incorporado. Ao identificar que o Cliente selecionou a opção Boleto, o caso de uso Pagar Compra é estendido pelo caso de uso Pagar com Boleto. Analogamente, caso o Cliente tenha selecionado a opção Cartão de Crédito, o caso de uso Pagar Compra será estendido pelo caso de uso Pagar com Cartão de Crédito.

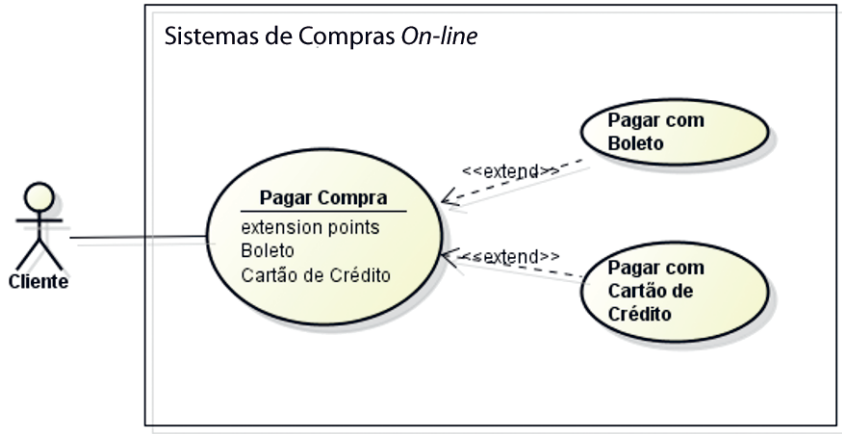


Figura 7. Relacionamento do tipo *extend* com pontos de extensão explícitos.

Um caso de uso pode ser utilizado para estender mais de um caso de uso. Da mesma forma, um caso de uso estendido pode também conter uma extensão. Em cada ponto de extensão apenas uma das opções de extensão será executada, ou seja, como se pode observar na Figura 7, ou o Cliente paga com Boleto ou o Cliente paga com Cartão de Crédito.

De acordo com a definição da UMLv2.5.1 (OBJECT MANAGEMENT GROUP, 2017, documento *on-line*),

> O caso de uso estendido é definido de forma independente do caso de uso que estende e faz sentido independentemente do caso de uso que estende. Por outro lado, o caso de uso que estende tipicamente define um comportamento que pode não necessariamente fazer sentido sozinho. Ao invés disto, o caso de uso que estende define um conjunto de incrementos de comportamento modulares que ampliam uma execução de um caso de uso estendido sob determinadas condições.

Relacionamento do tipo generalização

O relacionamento do tipo generalização funciona como o relacionamento de herança da orientação a objetos, ou seja, é utilizado quando o comportamento de um caso de uso (chamado de caso de uso geral) é herdado por outro caso de uso (chamado de caso de uso especializado). Os casos de uso especializados herdam todos os relacionamentos do caso de uso geral ao qual estão associados, mas a recíproca não é verdadeira.

O relacionamento de generalização é representado por uma linha contínua que inicia no caso de uso especializado e termina em uma seta aberta que aponta para o caso de uso geral. A Figura 8 apresenta um exemplo de generalização entre casos de uso em um exemplo parcial de um Sistema Bancário. Conforme se pode observar, um Cliente pode abrir uma Conta em um banco que pode ser uma Conta Corrente ou uma Conta Poupança. O comportamento do caso de uso Abrir Conta é especializado de acordo com o tipo da conta, executando os casos de uso Abrir Conta Corrente e Abrir Conta Poupança, conforme a seleção do Cliente.

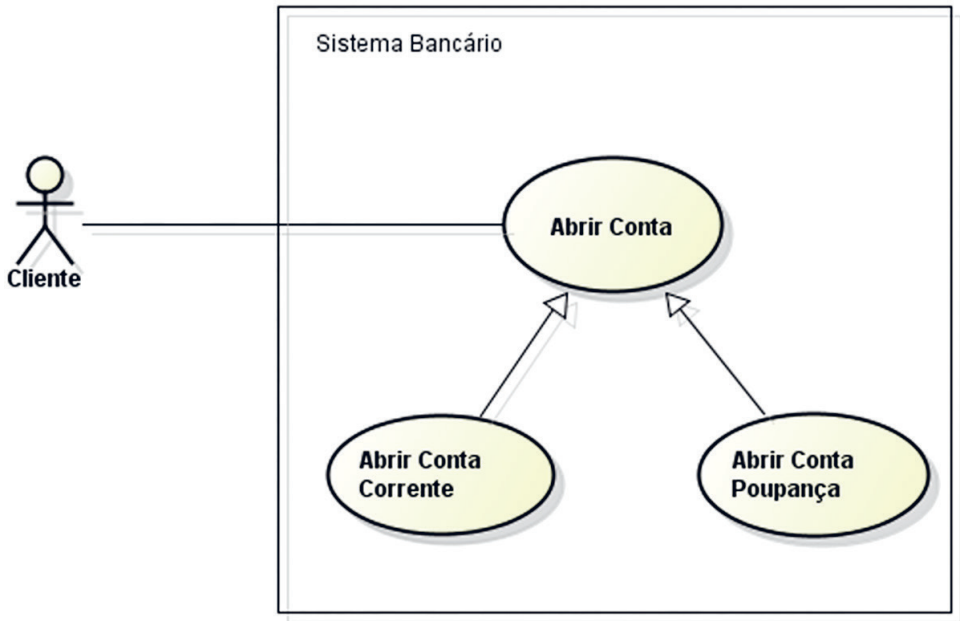


Figura 8. Relacionamento do tipo generalização de casos de uso.

O relacionamento do tipo generalização também é aplicável para os atores, e o conceito é exatamente o mesmo. Um ator especializado herda todos os casos de uso e os relacionamentos do ator geral.

Um exemplo pode ser visto na Figura 9, que mostra parcialmente um Sistema de Compras *On-line*. Um Visitante pode realizar o caso de uso Pesquisar Produtos, mas não pode executar o caso de uso Comprar Produtos. Ele só poderá Comprar Produtos quando se tornar um Cliente, que é um usuário cadastrado e logado. Por sua vez, um Cliente poderá tanto Pesquisar Produtos quanto Comprar Produtos.

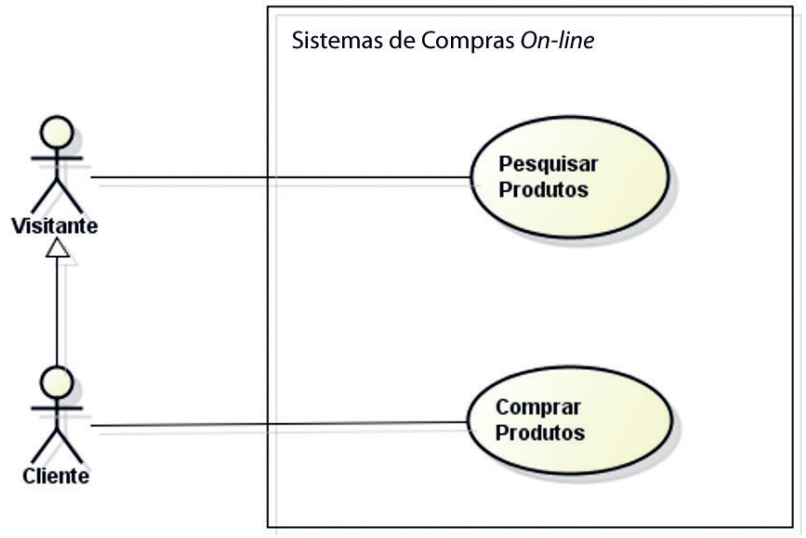


Figura 9. Relacionamento tipo generalização de atores.

O SWEBOK (*Software Engineering Body of Knowledge*) v3 (BOURQUE; FARLEY, 2014, p. 1–14) alerta o seguinte sobre os requisitos:

> Nem todas as organizações possuem a cultura de documentar e gerenciar os requisitos. É muito comum em empresas dinâmicas como *startups*, direcionadas por uma forte visão orientada a produto e recursos limitados, ver a documentação dos requisitos como um *overhead* desnecessário. Na maioria das vezes, no entanto, na medida em que estas empresas expandem, a sua base de clientes cresce, e o seu produto começa a evoluir, elas descobrem que necessitam recuperar os requisitos que motivaram as funcionalidades do produto de forma a avaliar o impacto das mudanças propostas. Consequentemente, a documentação dos requisitos e o gerenciamento de mudanças são a chave para o sucesso de qualquer processo de requisitos.

Referências

BOURQUE, P.; FAIRLEY, R. *SWEBOK:* guide to the *software* engineering body of knowledge version 3. [*S. l.*]: IEEE Computer Society, 2014.

DENNIS, A.; WIXON, B.; ROTH, R. *System analysis and design*. 5. ed. Hoboken: John Wiley and Sons, 2012.

GUEDES, G. *UML2*: uma abordagem prática. 3. ed. São Paulo: Novatec, 2018.

OBJECT MANAGEMENT GROUP. *Unified Modeling Language®*: OMG UML® v2.5.1. [*S. l.*], 2017. Disponível em: https://www.omg.org/spec/UML/About-UML/. Acesso em: 2 fev. 2020.

SEIDL, M. *et al. UML@Classroom:* an introduction to object-oriented modeling. Cham: Springer, 2012.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.

Parte 2

Aplicação do Diagrama de Casos de Uso

O conteúdo deste livro
é disponibilizado
por SAGAH.

SAGAH
SOLUÇÕES
EDUCACIONAIS
INTEGRADAS

Aplicação do diagrama de casos de uso

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Identificar a intenção dos atores primários em um cenário de negócio.
- Descobrir os requisitos funcionais de um *software*.
- Projetar um diagrama de casos de uso.

Introdução

Quando nos aproximamos de uma roda de desenvolvedores na hora do café, é comum que as conversas girem em torno da última tecnologia lançada, das tendências em *frameworks* e das ferramentas de desenvolvimento, se esta ou aquela linguagem de programação vai continuar fazendo sucesso, se é melhor usar este ou aquele padrão e assim por diante. Tecnologia é o que motiva o desenvolvedor!

No entanto, embora a tecnologia seja um aspecto muito relevante, se a equipe de desenvolvimento não conseguir capturar adequadamente as necessidades e os desejos do usuário, não importa o quão proficiente e experiente ela seja com a tecnologia, o sucesso do projeto estará comprometido. Compreender o que deve ser entregue sempre foi, e continua sendo, um grande desafio. Identificar requisitos de *software* não é uma atividade trivial. Elicitar requisitos exige um conjunto de habilidades técnicas e interpessoais que transcendem as habilidades com a tecnologia.

Diversos são os motivos pelos quais essa tarefa é tão desafiadora. Um deles é que não existe uma receita mágica que funcione em todas as situações. A quantidade de variáveis envolvidas é muito grande, como, por exemplo: tipo do *software* que está sendo desenvolvido, complexidade do *software*, experiência da equipe de desenvolvimento com a área de negócio, grau de inovação do produto ou serviço, perenidade da solução e habilidades interpessoais (os chamados *soft skills*) da equipe.

Essa dificuldade pode ser ainda maior quando o usuário já teve uma experiência ruim com um produto anterior ou, pior ainda, com o produto que está sendo desenvolvido ou modificado. Quando as suas expectativas já foram frustradas anteriormente, pode ser um desafio adicional para a equipe reverter a situação e curar as feridas.

Abordagens centradas no usuário têm sido a forma pela qual as organizações de tecnologia da informação (TI) têm buscado resolver essa questão. Entende-se por abordagem centrada no usuário aquela que coloca o usuário no centro do processo de desenvolvimento e foca na busca de valor agregado para ele. Este é um dos princípios das metodologias ágeis: entrega rápida de valor ao cliente.

Uma das primeiras abordagens desse tipo na área de TI, foi a utilização dos **diagramas de casos de uso** propostos por Ivar Jacobson. Por meio de elementos gráficos simples, o diagrama apoia a comunicação entre a equipe de desenvolvimento e o usuário na descoberta dos requisitos funcionais.

Mais recentemente, ferramentas desenvolvidas pela área de design têm sido também incorporadas como forma de apoiar o projeto de produtos e serviços que têm o usuário no centro do processo. Entre elas está o mapeamento da jornada do usuário, que busca identificar, não apenas as atividades que esse usuário realiza no processo, mas a forma como ele pensa e se sente ao realizar essas atividades, facilitando a identificação de elementos imprescindíveis no projeto de uma nova solução.

Neste capítulo você vai trabalhar com a aplicação do diagrama de casos de uso. Para isso vai ver como identificar as intenções dos atores ao utilizar o produto de *software* que está sendo desenvolvido, identificar os requisitos funcionais e por fim, projetar o diagrama de casos de uso. Também vai rever os conceitos envolvidos no diagrama de casos de uso e utilizar o mapeamento de personas e o *canvas* da proposta de valor para apoiar a sua construção.

1 Identificação da intenção dos atores

Vamos iniciar relembrando os conceitos do diagrama de casos de uso. O diagrama de casos de uso é um dos diagramas de comportamento da UML (GUEDES, 2018). Ele é usado para representar as funcionalidades do sistema, no formato de cenários ou casos de uso, conforme ilustrado na Figura 1:

- a fronteira do sistema, representada pelo retângulo que separa o meio externo e o meio interno do sistema;
- os atores, representados em forma de um boneco palito (*stick man*);
- os casos de uso, representados em forma de elipse;
- os relacionamentos entre os diversos elementos, representados pelas linhas.

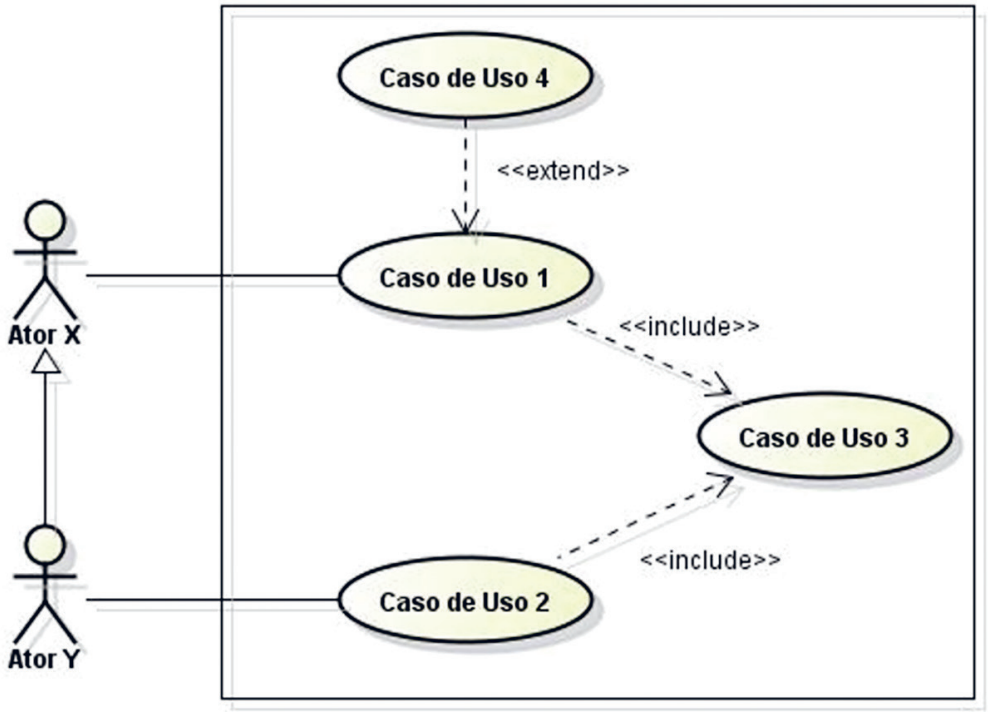


Figura 1. Diagrama de casos de uso.

O diagrama de casos de uso é útil para representar os requisitos funcionais sob a ótica dos usuários do produto de *software*. Compreender quais são as intenções dos usuários com o produto que está sendo construído é o primeiro passo para iniciar uma descoberta efetiva de requisitos. O que o usuário deseja não é o produto em si, mas os efeitos do uso do produto no seu dia a dia. O que ele quer é uma experiência de uso satisfatória e que atenda às suas necessidades, sejam elas no campo dos negócios, sejam para a vida pessoal e o lazer. O que o usuário deseja é a transformação que o *software* traz à sua realidade (PATTON, 2014).

Usar a nova solução proposta deve trazer mais satisfação ao usuário do que a solução que ele tem atualmente. Isso vale para qualquer situação, seja quando estamos desenvolvendo um produto para substituição de um produto existente, seja para uma solução inovadora e que irá prover um serviço ainda inexistente.

Naturalmente, nenhuma equipe de desenvolvimento tem o objetivo de criar uma solução que cause desapontamento e frustração ao cliente. Quando isso ocorre, não foi causado por uma ação deliberada ou premeditada pela equipe. Ninguém deseja entregar uma solução que não atenda ao cliente. Isso acontece porque um ou mais processos da equipe de desenvolvimento estão desalinhados. Frequentemente, esse processo é o de elicitação de requisitos.

Kalbach (2016) ressalta que os desalinhamentos, em um sentido mais amplo, impactam toda a organização: as equipes não têm um propósito comum; as soluções são construídas desconectadas da realidade; o foco maior está na tecnologia em vez de na experiência; e as estratégias são desenvolvidas com uma visão muito curta. Em contraste, organizações alinhadas têm um modelo mental compartilhado que aponta para onde querem chegar e são obcecadas pela entrega de experiências extraordinárias para os seus clientes.

Colocar o usuário no centro do processo é uma das formas de evitar o desgaste de entregas insatisfatórias. Compreender as intenções das classes de usuários e mapeá-las como intenções dos atores que irão interagir com o *software* é uma das formas de reduzir a distância entre as expectativas e as entregas.



Fique atento

Fique atento ao que Jim Kalbach (2016) sugere sobre os pontos imperativos para que o alinhamento possa ocorrer:

- **foque as suas ofertas de fora para dentro em vez de dentro para fora** — todos devem ter empatia com aqueles e quem servem; os indivíduos da organização devem se importar profundamente com os clientes e suas experiências, devem internalizar os desejos e as motivações das pessoas e devem ser defensores dos interesses delas em todas as suas atividades.
- **alinhe todas as funções internas em todos os níveis** — os processos internos da organização estão tão relacionados com a experiência do usuário quanto os pontos visíveis da interação. Organizações alinhadas não focam em partes da experiência, mas sim na experiência como um todo.
- **crie visualizações como visão compartilhada** — artefatos visuais ajudam a criar uma visão compartilhada. Eles não garantem o alinhamento, mas provocam discussões que promovem o alinhamento. Representações visuais são o *must have* (precisa ter) do alinhamento estratégico.

O **mapeamento visual** ajuda a compreender sistemas complexos de interação, especialmente quando conceitos abstratos como experiência estão envolvidos. Mapear as experiências não é uma atividade que está limitada a um único diagrama, existem muitas perspectivas e abordagens (KALBACH, 2016). Neste capítulo optamos por abordar o **diagrama de casos de uso** (Figura 1) como ferramenta de representação das funcionalidades do sistema e da interação entre os atores e essas funcionalidades e o ***canvas* da proposta de valor** como forma de identificar as expectativas e experiências do usuário, de modo a obter as intenções dos atores. Juntas, essas ferramentas visam a apoiar o desenvolvimento centrado no usuário.

Definição da proposta de valor

De acordo com a UML (OBJECT MANAGEMENT GROUP, 2017, documento *on-line*), "Um ator especifica um papel que é desempenhado por um usuário ou qualquer outro sistema que interage com o sujeito". **Sujeito**, neste caso, é o sistema que está sendo desenvolvido. **Ator**, portanto, pode ser um agente humano, que interage com o sistema, ou também outro sistema (que pode ser um *hardware*, outro *software* ou uma combinação deles). Para projetar um diagrama de casos de uso, inicialmente é preciso descobrir quem são os atores e que intenção eles têm com o sistema.

O primeiro passo para a definição da intenção dos atores é começar identificando a **proposta de valor**. Valor é aquilo que o cliente percebe como importante para ele e pelo qual está disposto a pagar. Valor não é o preço que o produto vai custar, valor é a relevância, a importância que o produto terá na vida do cliente.

Uma das formas de se mapear o valor é utilizando o modelo ***canvas* de proposta de valor**. A definição da proposta de valor é um dos elementos que fazem parte do ***canvas*** do **modelo de negócios**, proposto originalmente por Osterwalder (2004). Posteriormente, a proposta de valor foi destacada pelo mesmo autor em um diagrama próprio, conforme ilustrado na Figura 2.

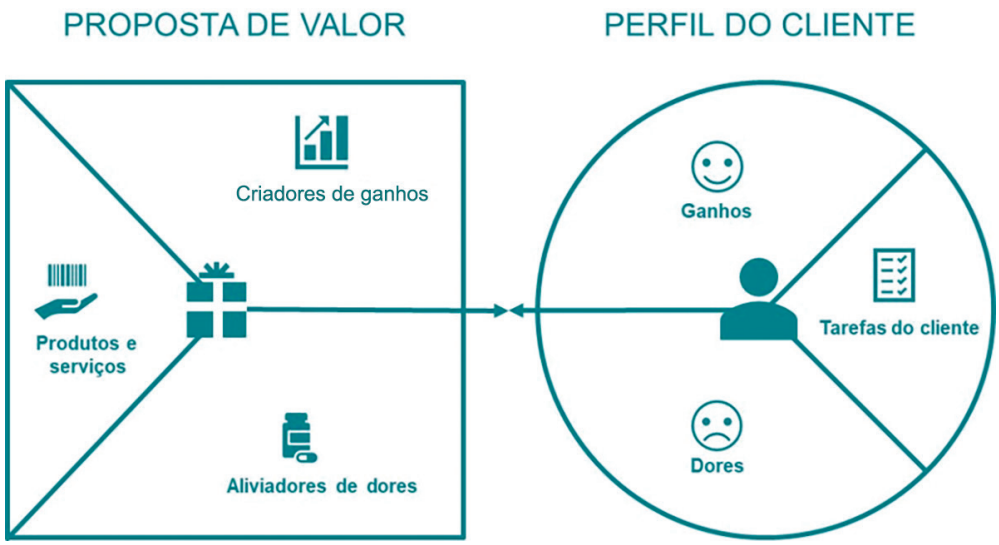


Figura 2. *Canvas* de proposta de valor.

Do lado direito do diagrama fica a parte relacionada ao perfil do cliente e, do lado esquerdo, a parte relacionada à solução, ou seja, à proposta de valor. Como se pode observar, o objetivo é promover o alinhamento entre as expectativas do cliente, os produtos e serviços providos pela organização. Esse objetivo de alinhamento é representado pelas setas que saem de uma figura em direção à outra.

Esse modelo é muito versátil e pode ser utilizado em diversos tipos de situação, desde a modelagem do negócio de uma organização em si até a modelagem para a construção de um produto de *software*, como é o caso que utilizaremos neste capítulo. Esse diagrama também tem sido muito empregado nas fases iniciais de concepção e criação de *startups*. Nossa proposta é que ele sirva como um passo inicial antes da construção do diagrama de casos de uso.

E como esse diagrama é construído? Essencialmente em parceria com os *stakeholders* relevantes do projeto, em especial os usuários do produto que está sendo construído. Isto pode acontecer em uma seção de *brainstorming* ou *workshop* de requisitos. A forma mais prática é imprimir uma versão grande do diagrama (formato A0, por exemplo), que possa ser colada em uma parede e que permita que todos tenham acesso a ela para escrever suas contribuições em notinhas adesivas tipo Post-it®, que são fáceis de serem removidas, descartadas, alteradas ou realocadas entre as seções do mapa. É comum que uma cor de Post-it seja utilizada para cada item do diagrama, facilitando a sua leitura.

O objetivo é que os participantes possam gerar o *canvas* de proposta de valor de forma participativa e colaborativa, de modo a estreitar o seu entendimento sobre as dores dos usuários e como a solução desenhada poderá trazer alívio para essas dores, concretizando os ganhos esperados.

O lado direito do diagrama diz respeito ao **espaço do problema**, ou seja, o **perfil do cliente**. Esse espaço é subdividido nas três seções, apresentadas a seguir,

- **Tarefas do cliente (*jobs*):** quais são as tarefas funcionais, sociais ou emocionais do lado do cliente.
- **Dores (*pains*):** são aspectos adversos das tarefas do cliente que ele espera poder evitar — resultados indesejados, obstáculos e riscos.
- **Ganhos (*gains*):** quais são os benefícios que o cliente deseja alcançar, quais são os ganhos financeiros, sociais, emocionais obtidos por meio da realização das tarefas.

O lado esquerdo do diagrama diz respeito ao **espaço da solução**, ou seja, o **mapa de valor**, que é subdividido em três seções também, listadas a seguir.

- **Produtos e serviços (*products and services*):** a solução que está sendo construída.
- **Aliviadores de dores ou analgésicos (*pain relievers*):** como resolver a dor do cliente? O que elimina a dor do cliente, seu estresse ou suas emoções negativas sobre o produto ou serviço?
- **Criadores de ganho (*gain creators*)**: o que o produto ou serviço oferece de ganho para o cliente, seja em termos de redução de custos, seja em emoções positivas ou ganho social?

Gerar o *canvas* da proposta de valor é mais que fazer um diagrama, é favorecer a comunicação entre os *stakeholders* e a equipe de desenvolvimento por meio de uma ferramenta gráfica simples, em uma interação que visa à cocriação para atingir um entendimento comum sobre o produto.

Exemplo

Veja aqui um exemplo do modelo *canvas* de proposta de valor, considerando o caso da Netflix. O cliente, do lado direito, deseja uma forma de assistir filmes e séries em diversas localidades, com qualidade e baixo custo. O provedor oferece serviço de *streaming* de vídeo *on demand*.



Algumas dicas podem ajudar na construção do *canvas* da proposta de valor. Veja a seguir.

- Comece identificando as personas que utilizarão o seu produto (personagens que representam uma classe real de usuários do sistema).
- É possível, na mesma sessão, criar mais que um *canvas* de proposta de valor, cada um sob a perspectiva de uma persona. Pode-se, inclusive, ter grupos distintos trabalhando simultaneamente, cada um com um tipo de persona.
- Prepare a sessão presencial com antecedência para que todas os *stakeholders* essenciais para o seu produto possam estar presentes.
- Prepare uma lista de perguntas que possam apoiar as reflexões dos participantes, para o caso de eles não conseguirem evoluir sozinhos.

- Incentive fortemente que todos participem e coloquem sua opinião no quadro — especialmente suas dores e sua percepção sobre os ganhos que desejam.
- Assim como em uma sessão de *brainstorming*, desestimule as críticas às colocações dos colegas para que todos possam se expressar livremente, sem o receio de serem criticados ou ridicularizados.
- Ao final, faça uma rodada para a leitura e esclarecimento de todos os Post-it e para garantir o entendimento comum do que está registrado.
- Se os participantes permitirem, a rodada final de leitura dos Post-it poderá ser gravada, de forma a facilitar a compilação e análise posteriormente.
- Tenha em mente que o objetivo não é a construção do mapa em si, mas chegar a um entendimento comum sobre o que está sendo desenvolvido, por que está sendo desenvolvido e o que se espera como efeitos do uso do produto.



Link

Para elaborar o *canvas* de modelo de negócios, acesse o *link* a seguir e utilize a cartilha elaborada pelo SEBRAE especialmente para explicar o uso dessa ferramenta.

https://qrgo.page.link/pWvX1

Intenção dos atores primários

Apresentamos, nas subseções anteriores, uma das ferramentas que vem sendo muito utilizada para ajudar a aprofundar o entendimento das necessidades de quem utilizará o produto que está sendo desenvolvido, ou seja, os atores humanos. Mas não esqueça que existem também os atores não humanos, como *hardware* e *software* com os quais o nosso sistema irá interagir. Vivemos em um mundo tecnologicamente integrado e é pouco provável que você venha a desenvolver um produto de *software* que não precise conversar com outro para realizar as suas funções de forma completa. Em tempos de internet das coisas (IoT, do inglês, *internet of things*) e equipamentos e serviços distribuídos, é importante identificar e compreender as necessidades dessas interfaces logo no início do projeto.

2 Descoberta dos requisitos funcionais

Como você observou na seção anterior, definir a intenção dos atores é um dos primeiros passos para chegar aos requisitos funcionais, ou seja, às funcionalidades que devem ser providas pelo produto de *software* para atender a essas intenções.

Existem diversas formas de elicitar os requisitos, e a escolha da técnica mais adequada vai depender do contexto do projeto, ou seja, do tipo de produto, do negócio, da disponibilidade de informações, da disponibilidade de fontes para obter essas informações, da experiência da equipe de desenvolvimento e mais uma dezena de aspectos particulares de cada projeto.

Algumas das técnicas mais utilizadas são:

- entrevista;
- reunião tradicional;
- reunião no formato de JAD (*joint application design*);
- *brainstorming*;
- questionário;
- observação;
- análise de sistemas anteriores;
- análise de leis e regulamentações.

Não vamos discutir as técnicas de elicitação de requisitos neste capítulo, mas tenha em mente que você pode combiná-las para chegar ao melhor resultado. Wiegers e Beatty (2013) oferecem boas dicas para apoiar essa seleção.

Ao elicitar os requisitos, é comum que eles venham em diferentes tipos e níveis de abstração. Alguns estarão relacionados ao produto de *software* em si, enquanto outros trarão informações em relação a restrições de projeto ou de processo, como data esperada de conclusão do projeto, uso de determinada tecnologia, uso de determinado processo e desenvolvimento e assim por diante. Mesmo os requisitos de produto podem vir misturados entre os requisitos funcionais e os não funcionais (como desempenho, usabilidade, manutenibilidade, etc.)

O diagrama de casos de uso é excelente para elicitar os requisitos funcionais, ou seja, as funções que o usuário necessita e que realizará por meio do produto de *software*. Histórias do usuário, utilizadas nos métodos ágeis, assemelham-se muito aos casos de uso, exceto que não têm uma forma gráfica de representação.

3 Como projetar o diagrama de casos de uso

O diagrama de casos de uso utiliza uma linguagem gráfica simples, composta por símbolos fáceis de rabiscar à mão livre, conforme vimos na Figura 1 no início deste capítulo. Apesar desta simplicidade, e talvez até por causa dela, ele possui um alto poder de comunicação. É fácil para um usuário leigo se ver realizando as tarefas representadas pelos casos de uso e apoiar a sua construção.

O diagrama de casos de uso se aplica aos requisitos funcionais, mas não foi criado para tratar os requisitos não funcionais.

Nível de abstração

Uma das primeiras dúvidas que surgem quando se começa a trabalhar com casos de uso é sobre o nível de abstração de um caso de uso. Até que nível se deve chegar no detalhamento de um caso de uso? E a resposta é: depende! À medida que você for usando essa ferramenta, vai sentir se detalhou de menos ou demais, e vai promovendo os ajustes para os próximos projetos ou as próximas *sprints*.

Para começar, você pode se basear na dica de um dos precursores dos métodos ágeis, Alistair Cockburn (2005). Ele considera três níveis de abstração para os objetivos de uma tarefa, utilizando uma metáfora da altitude, conforme ilustrado na Figura 3.

Um objetivo está no nível do mar quando se refere a uma tarefa que você vai realizar sem interrupção, ou seja, aquela que você "realiza em uma sentada". Por exemplo, comprar um produto pela internet. Patton (2014) exemplifica dizendo que é o equivalente a uma tarefa do tipo tomar banho: você não para no meio do banho para tomar um cafezinho e voltar depois. Então essa é uma tarefa que está no nível do mar, ou seja, uma tarefa no nível do usuário. Cockburn (2005) diz que esse é um objetivo azul (*blue goal*), chama essas tarefas de tarefas em nível funcional e as representa como uma onda do mar. Todo o restante ou está acima ou abaixo do nível do mar.



Figura 3. Níveis de abstração dos casos de uso.

Fonte: Adaptada de Cockburn (2005).

Cockburn (2005) considera que um objetivo está acima do nível do mar quando tem diversos objetivos no nível do mar a ele relacionados. Isso ocorre em três situações:

- mostrar o contexto no qual o objetivo opera;
- mostrar o ciclo de vida que os objetivos relacionados operam;
- servir como uma espécie de índice para os objetivos abaixo dele.

Ele ainda divide esse nível em dois tipos: **super sumário** (nível bem abstrato) e **sumário** (nível abstrato). O primeiro é representado por uma nuvem e considerado como super branco (*very white goal*). O segundo é representado por uma pipa e considerado como branco (*white goal*).

Por fim, Cockburn (2005) considera que os objetivos que estão abaixo no nível do mar são as **subfunções** (*indigo goals*). Geralmente não precisamos listá-los, a menos que seja absolutamente necessário em termos de clareza e compreensão do escopo. A esses objetivos ele atribui a cor índigo e os compara com peixes. Ele brinca que existem casos de uso que são tão detalhados que estão no fundo do mar, como mariscos ou ostras, e são da cor preta (*black goals)*. Estes não deveriam nem existir.

Diagrama de caso de uso na prática

A melhor forma de discutirmos as peculiaridades deste diagrama é por meio de um exemplo. Vamos ver um deles para ajudar a tirar as dúvidas sobre a sua construção. A Figura 4 apresenta o diagrama de casos de uso do aplicativo Livros *On-line*, que funcionará para a compra e leitura de livros de forma digital, como faz o aplicativo Kindle, por exemplo. Trataremos aqui apenas da parte que oferece as funcionalidades para o leitor e que será usada em dispositivos móveis (celulares e *tablets*). A parte de *backoffice*, que administra o negócio, não será tratada neste exemplo.

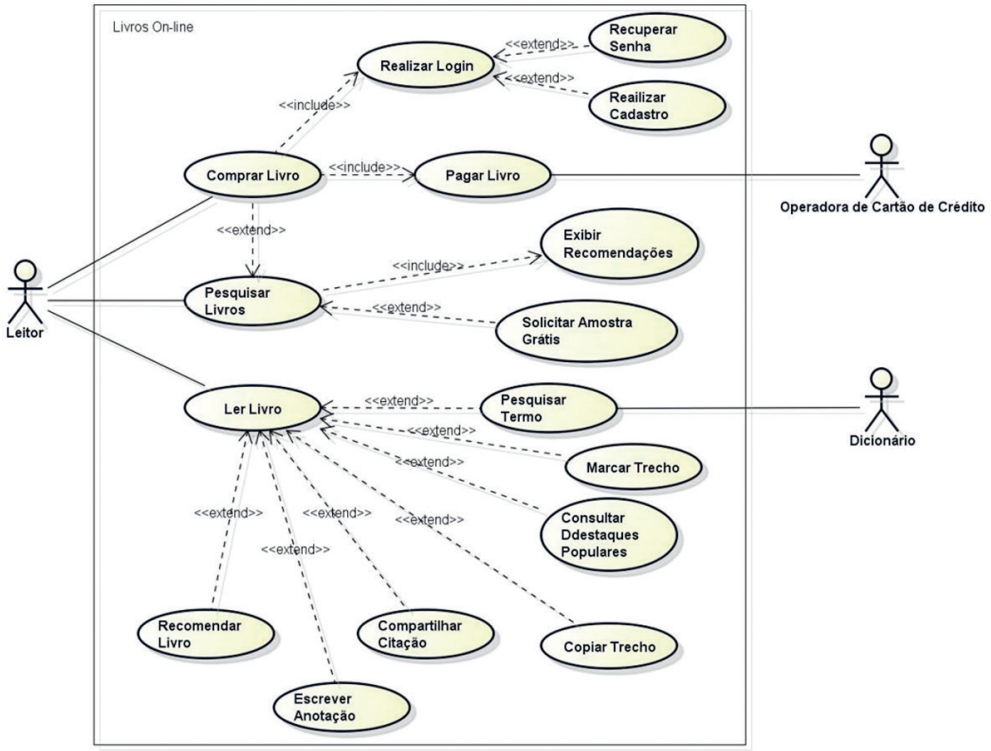


Figura 4. Diagrama de casos de uso do aplicativo Livros *On-line*.

Vamos começar pela **fronteira do sistema**. Um retângulo separa a parte interna do sistema da parte externa. O nome do sistema é colocado na parte superior desse retângulo. Os atores ficam do lado de fora, uma vez que são agentes externos ao sistema.

Inicialmente, identificamos três **atores**: o Leitor (que é o **ator principal** desse aplicativo), a Operadora de Cartão de Crédito (por meio da qual serão feitos os pagamentos) e o Dicionário (que é um sistema externo que, como o próprio nome diz, provê as funções de dicionário, incluindo a tradução de termos). Tanto a Operadora de Cartão de Crédito quanto o Dicionário são considerados **atores secundários**. Geralmente, colocam-se os atores principais do lado esquerdo do diagrama e os atores secundários do lado direito.

Note que não chamamos o Leitor de Cliente ou de Usuário, pois esses são nomes genéricos e pouco significativos. É recomendado optar por nomear os atores com um substantivo mais significativo para o contexto. Neste caso, o leitor.

Para que os requisitos desse Leitor possam ser adequadamente definidos podemos iniciar mapeando o perfil de leitores que, possivelmente, vão se interessar pelo aplicativo. Isso pode ser feito com o **mapeamento das personas**.

Vamos pensar em dois perfis distintos:

- leitores que buscam por leitura técnica (não ficção);
- leitores que buscam por leitura não técnica (ficção).

No primeiro perfil podemos pensar em professores, universitários, pesquisadores e profissionais. Esse perfil pode desejar funcionalidades que facilitem suas pesquisas, como copiar uma citação e carregar junto com ela a referência do livro.

Já um perfil de uso doméstico, por exemplo, que utiliza o livro para leituras de lazer (livros de ficção), não necessita de uma função como essa, mas pode desejar compartilhar suas preferências com listas de leitores de um mesmo gênero, por exemplo, ou até mesmo montar um clube de leitura.

Pensar nos diferentes perfis nos ajuda a identificar requisitos funcionais ou não funcionais para essas classes específicas de usuários. Se quisermos atingir, por exemplo, um público com alguma deficiência visual, teremos que contemplar elementos na interface que facilitem a leitura para esse público, como aumento das letras ou até mesmo a leitura em áudio. Uma persona com essas características precisa ser então projetada.

Em seguida, identificamos **três casos de uso principais** que o Leitor pode realizar: ele pode Pesquisar Livros, Comprar Livros e Ler Livros. Note que todos eles estão diretamente ligados ao ator principal, o Leitor. Esses casos de uso podem ser considerados, na visão de Cockburn (2005), como os do nível do mar.

Essas funcionalidades podem ser provenientes de diversas fontes, como já discutimos anteriormente neste capítulo. Uma delas pode ser o *workshop* para o mapeamento do *canvas* da proposta de valor. Geralmente diversas técnicas combinadas são usadas para essa finalidade.

Quando o Leitor estiver pesquisando um livro, ele pode, opcionalmente, solicitar uma amostra grátis. Isso é representado pelo **relacionamento *extend*** entre o caso de uso Pesquisar Livro e Solicitar Amostra Grátis. Neste mesmo ponto temos um **relacionamento *include*** entre Pesquisar Livro e Exibir Recomendações. Isso significa que toda vez que uma pesquisa for feita, serão exibidas recomendações similares à pesquisa.

Qualquer Leitor pode Pesquisar Livros, mas, para comprar, esse Leitor deverá estar logado. Isso é representado pelo relacionamento *include* que existe entre Comprar Livro e Realizar Login. Se ele não tiver um cadastro, terá que Realizar Cadastro (relacionamento de *extend*). Se ele esqueceu a senha, pode acionar o Recuperar Senha (outro relacionamento de *extend*).

A compra do livro é paga com cartão de crédito e dois atores são necessários para concluir essa transação: o Leitor e a Operadora de Cartão de Crédito. Caso houvesse mais que uma forma de pagamento, então seria possível utilizar o **relacionamento de generalização** para contemplar as demais formas.

O terceiro caso de uso principal que o ator Leitor pode executar é Ler Livro. Essa é a funcionalidade principal do aplicativo e a ela precisa ser dedicada toda a atenção no momento de elicitação dos requisitos, para que sejam incorporadas funcionalidades que agreguem valor a cada um dos perfis que definimos (personas). Todos os relacionamentos com os casos de uso Ler Livro são do tipo *extend*, uma vez que não são obrigatórios, ou seja, o usuário pode ou não executar aquelas subfunções. Estão listados, neste caso: Recomendar Livro, Escrever Anotação, Compartilhar Citação, Copiar Trecho, Consultar Destaques Populares, Marcar Trecho e Pesquisar Termo.

Note que, para que o caso de uso Pesquisar Termo possa ser realizado, ele precisa do sistema externo Dicionário. Portanto, não se trata de uma funcionalidade implementada internamente, mas de uma interface com um sistema externo. Isso é bastante comum no desenvolvimento de *software*, uma vez que é difícil termos sistemas isolados e que não se integram com outros.

Neste exemplo específico não tivemos a necessidade de mapear um ator do tipo *hardware* porque não temos nenhum *hardware* especial envolvido. Se tivéssemos, ele seria mapeado como um ator, utilizando a mesma simbologia (boneco palito), ou poderia usar um ícone (SEIDL *et al.*, 2012). Utilizar ícones não é muito comum em diagramas de caso de uso na prática, mas é permitido pela UML (OBJECT MANAGEMENT GROUP, 2017).

Caso o nosso exemplo tivesse uma distinção entre um Leitor comum e um Leitor VIP, no qual houvesse algum tipo de privilégio para o Leitor VIP, então, a forma adequada de representar seria por meio do relacionamento de generalização, conforme demonstrado na Figura 5.

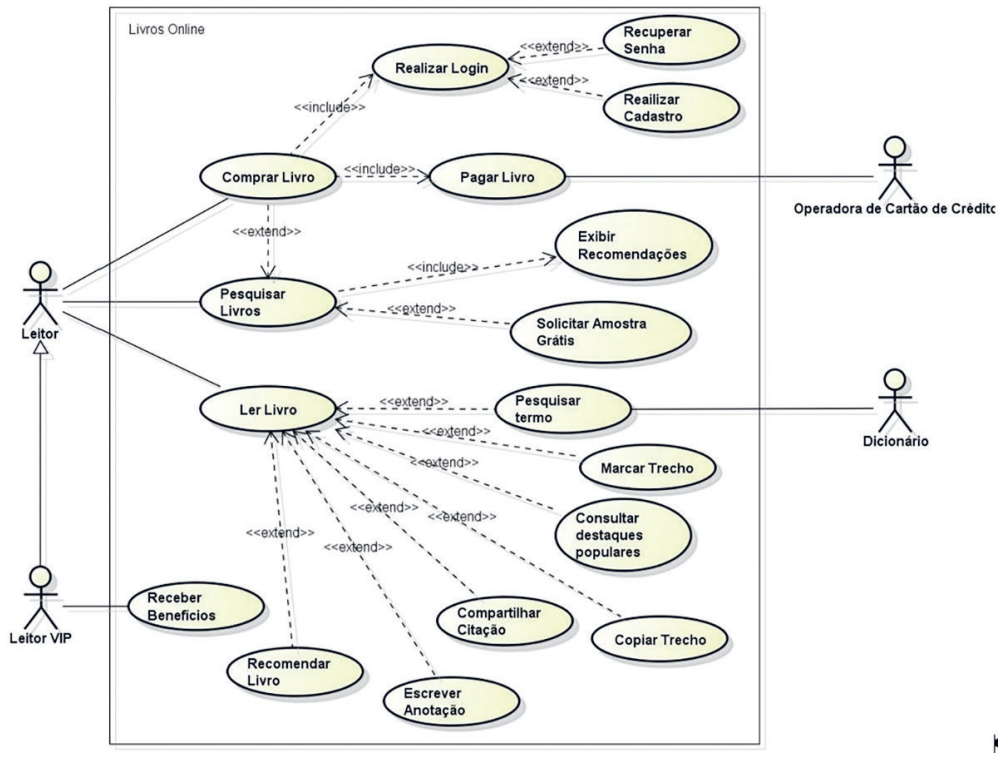


Figura 5. Diagrama de casos de uso do aplicativo Livros *On-line* com Cliente VIP.

Como se pode observar pela Figura 5, o Cliente VIP pode fazer tudo o que o Cliente faz. Ele herda todos os relacionamentos do Cliente normal e pode realizar um caso de uso adicional, que só ele pode realizar, que é Receber Benefícios. Essa é uma das formas de se representar este tipo de situação.

Esta situação aqui foi implementada no diagrama criando um novo tipo de ator, o Cliente VIP e um novo caso de uso, Receber Benefícios. Outra forma de fazer esta representação seria não inserir o ator adicional Cliente VIP, mas adicionar o caso de uso Receber Benefícios como um *extend* de outro caso de uso, em que o benefício fosse ser associado. Por exemplo, se ele tem direito a baixar alguns livros gratuitamente, esse caso de uso poderia estar relacionado com o caso de uso Pagar Livro.

Referências

COCKBURN, A. *Escrevendo casos de uso eficazes*: um guia prático para desenvolvedores de software. Porto Alegre: Bookman, 2005.

GUEDES, G. *UML2*: uma abordagem prática. 3. ed. São Paulo: Novatec, 2018.

KALBACH, J. *Mapping experiences*: a guide to creating value through journeys, blueprints, and diagrams. Beijin: O´Reilly, 2016.

OBJECT MANAGEMENT GROUP. *Unified Modeling Language®:* OMG UML® v2.5.1. [*S. l.*], 2017. Disponível em: https://www.omg.org/spec/UML/About-UML/. Acesso em: 2 mar. 2020.

OSTERWALDER, A. *The business model ontology*: a proposition in a design science approach. 2004. Tese (Doutorado)- Université de Luasanne, Lausanne, 2004. Disponível em: http://www.hec.unil.ch/aosterwa/PhD/Osterwalder_PhD_BM_Ontology.pdf. Acesso em: 2 mar. 2020.

PATTON, J. *User story mapping:* discover the whole story, build the right product. Sebastopol: O´Reilly, 2014.

SEIDL, M. *et al*. *UML@Classroom:* an introduction to object-oriented modeling. Cham: Springer, 2012.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.

Leitura recomendada

SEBRAE. *Cartilha O quadro de modelo de negócios*: um caminho para criar, recriar e inovar em modelos de negócios. Brasília, DF: SEBRAE, 2013. Disponível em: www.sebrae.com.br/Sebrae/Portal%20Sebrae/UFs/ES/Anexos/ES_QUADROMODELODENEGOCIOS_16_PDF.pdf. Acesso em: 02 mar. 2020.

Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Parte 3

Especificação de Requisitos Funcionais Utilizando Casos de Uso

O conteúdo deste livro é disponibilizado por SAGAH.

SAGAH SOLUÇÕES EDUCACIONAIS INTEGRADAS

Especificação de requisitos funcionais utilizando casos de uso

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Identificar os elementos que compõem uma especificação de caso de uso.
- Criar o fluxo principal de um cenário de caso de uso.
- Projetar fluxos alternativos e de exceção em cenários de caso de uso.

Introdução

Há várias formas de expressar os requisitos funcionais de um projeto. Podemos fazer isso por meio de desenhos, diagramas, textos escritos em linguagem natural, cartões, Post-it® ou qualquer outro meio que a equipe considere eficiente. O mais importante é a essência, ou seja, que tenha sido estabelecida a conexão, o alinhamento, entre a equipe de desenvolvimento e o cliente. A forma como isso será registrado vai depender de diversas variáveis de contexto.

Independentemente de o ambiente no qual o projeto está sendo desenvolvido ser gerenciado de forma mais tradicional ou de forma ágil, requisitos sempre serão o ponto fundamental do sucesso do projeto. Compreender a real necessidade do negócio e projetar a forma como o *software* pode agregar valor nesse contexto já é meio caminho andado.

Casos de uso têm sido uma forma pela qual as organizações desenvolvedoras de *software* têm especificado requisitos funcionais. Isso é feito utilizando-se tanto o diagrama de casos de uso, proveniente da UML (em inglês, *Unified Modeling Language*), quanto a especificação de casos de uso, que tem origens e formatos diversos.

Neste capítulo você vai estudar os elementos que compõem a especificação de casos de uso, uma importante ferramenta de apoio ao diagrama de casos de uso. Vai ver também como identificar os passos que compõem o fluxo principal de um caso de uso, os fluxos alternativos e de exceção.

1 Elementos da especificação de casos de uso

Especificar requisitos funcionais utilizando a abordagem de casos de uso requer duas etapas, uma se refere à construção do diagrama de casos de uso e outra, à especificação que acompanha o diagrama. Pensar em casos de uso significa colocar o usuário no centro do processo e definir funcionalidades que agreguem valor para esse usuário.

Diagrama de casos de uso

O diagrama de casos de uso é uma ferramenta visual que representa o comportamento do sistema sob a ótica do usuário (SEIDL *et al.*, 2012). Ele oferece uma visão geral das funcionalidades de um sistema que envolve *software*. Qualquer *stakeholder*, mesmo leigo, consegue fazer a leitura de um diagrama de casos de uso e compreender o seu significado, bastando conhecer a sua simbologia, que é bastante simples. O diagrama de casos de uso não oferece uma visão interna da implementação. Para isso existem outros diagramas da UML que podem ajudar.

Vamos relembrar o diagrama de casos de uso, tomando como base o contexto mapeado na Figura 1. Suponha que você acaba de entrar em uma empresa e que o seu novo gerente lhe passa este diagrama e pede que você vá se familiarizando com ele para que vocês possam conversar no final da manhã. Mesmo sem conhecer os detalhes deste *software*, você já consegue rapidamente identificar quem são os atores que fazem parte do contexto, bem como quais são as principais funcionalidades envolvidas. A partir desta representação, já é possível inferir também algumas regras de funcionamento do sistema. Se você tiver alguma experiência, vai até conseguir descobrir que há coisas faltando no diagrama e identificar questões que você poderá esclarecer com o seu gerente quando vocês se reunirem.

Ao observar este diagrama, é possível identificar que se trata de um Sistema de Compras *On-line*, uma vez que existe um retângulo que define a fronteira da aplicação e um título no canto superior esquerdo que indica o nome do sistema.

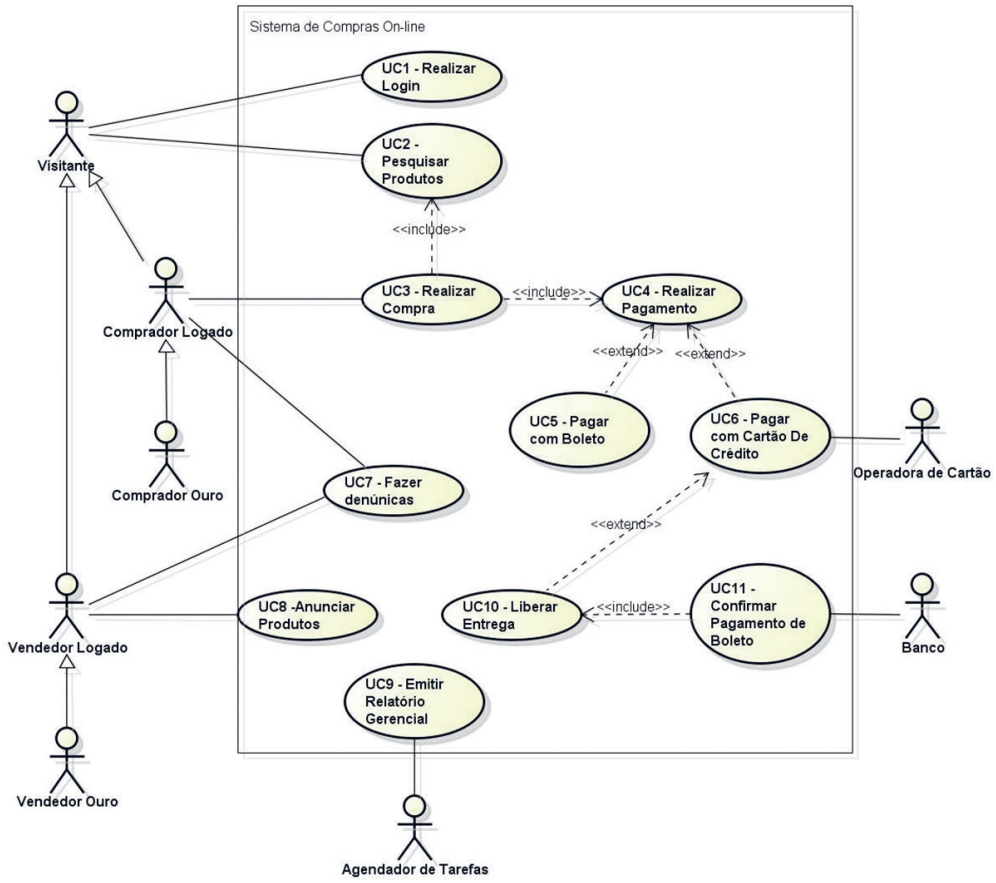


Figura 1. Diagrama de casos de uso.

É possível ainda identificar que qualquer pessoa pode acessar o sistema como visitante e pesquisar produtos, mas, para comprar um produto, é preciso estar logado. Um Comprador Logado poderá realizar todas as funções que um Visitante executa, mas o contrário não é verdadeiro. Você sabe disso porque viu que há um relacionamento de Generalização, que denota a herança entre esses dois atores (a seta sai do Comprador e aponta para o Visitante).

Neste momento você percebe que existe uma diferença entre os compradores, pois há a figura do Comprador Ouro, e se pergunta: "Mas o que este Comprador Ouro tem de especial? Ele tem algum benefício adicional? Não consigo ver isto no diagrama. Vou perguntar para o gerente". É possível identificar a mesma situação com o papel do vendedor, ou seja, existe um Vendedor Ouro, mas ainda não sabemos qual é a diferença dele em relação ao ator Vendedor.

Você percebe que ao realizar uma compra existem dois tipos de pagamento diferentes, o que é demostrado pelos relacionamentos de *extend* existentes entre o caso de uso UC4 – Realizar Pagamento e os casos de uso UC5 – Pagar com Boleto e UC6 – Pagar com Cartão de Crédito. Note ainda que, quando for feito o pagamento com cartão de crédito, outro ator terá que ser envolvido, a Operadora de Cartão, uma vez que há uma associação entre eles. Aqui você também já anota que esta situação poderia também ter sido representada por uma relação de Generalização entre UC4 – Realizar Pagamento e os seus possíveis herdeiros UC5 e UC6.

Você percebe que a liberação da entrega depende da confirmação do pagamento. No caso do pagamento com cartão de crédito, a confirmação é *on-line* com a Operadora de Cartão. No caso do pagamento com boleto, isso vai acontecer quando o ator Banco confirmar o pagamento. Isso é representado pelo relacionamento de *include*, que indica que, quando o caso de uso UC11 – Confirmar Pagamento de Boleto for executado, ele chamará o caso de uso UC10 – Liberar Compra para ser executado também.

Você continua sua análise e descobre que existe o caso de uso UC7 – Fazer Denúncias, mas nota que não existe um caso de uso para tratar a denúncia e responder ao denunciante. Faz então mais uma anotação para perguntar ao seu gerente: "Será que a denúncia será tratada manualmente? Teremos que nos relacionar com algum outro sistema? Isso não está claro!".

Lá no final do diagrama você nota que tem um ator chamado Agendador de Tarefas e se lembra que esse tipo de ator é usado quando existem casos de uso que são disparados automaticamente pelo próprio sistema, em resposta a eventos temporais ou de controle. Neste caso, o sistema vai disparar o caso de uso UC9 – Emitir Relatório Gerencial. Aí você pensa: "Qual será a periodicidade que isto ocorre? Poderia ter sido adicionada uma nota ao diagrama, descrevendo em que condições este relatório será emitido". Então acrescenta mais uma pergunta à sua lista de questões a serem discutidas com o seu gerente.

Para finalizar sua análise você observa que existe um caso de uso UC1 – Realizar Login, mas não existe nada para tratar situações de exceção, como o esquecimento de senha ou o usuário não estar cadastrado. Você decide questionar seu gerente como será tratada essa questão: se for com um sistema externo, está faltando representá-lo no diagrama; se for pelo próprio sistema, então faltam novos casos de uso, como Realizar Cadastro e Recuperar Senha.

Mesmo você não tendo participado da elicitação de requisitos desse *software*, apenas com essa rápida análise, já foi possível perceber as principais funcionalidades, além de levantar algumas dúvidas que ainda precisam ser esclarecidas.

O diagrama de casos de uso realmente é simples e tem um grande poder de comunicação. No entanto, apenas o diagrama não é suficiente para dar continuidade às demais etapas do ciclo de vida de desenvolvimento, uma vez que faltam detalhes para que as funcionalidades possam ser implementadas com sucesso pelos desenvolvedores e testadas pelos testadores. Esses detalhes fazem parte do que é chamado de especificação de casos de uso.



Fique atento

Como você já percebeu, cada elemento do diagrama tem um significado específico, e ele precisa estar coerente com os elementos que compõem a especificação do caso de uso. Eles formam um **conjunto coerente**, no qual o diagrama de casos de uso representa a visão geral das funcionalidades de forma gráfica e a especificação de casos de uso apresenta os detalhes da execução dessas funcionalidades. Esses dois elementos precisam estar **alinhados**. Se um for alterado, é necessário analisar se o outro também não sofrerá modificações.

Especificações de casos de uso

Uma especificação de casos de uso consiste em um detalhamento dos cenários de execução dos casos de uso. São acrescentadas informações àquelas que foram representadas graficamente no diagrama, de modo que a equipe de desenvolvimento tenha mais subsídios para validar e seguir para a implementação.

A primeira dúvida que surge quando se inicia a especificação de casos de uso é: até que nível devemos detalhar? E a resposta é: depende. Depende do grau de proximidade dos usuários com a equipe de desenvolvimento, depende do grau de maturidade da própria equipe de desenvolvimento, depende do grau de intimidade que a equipe de desenvolvimento tem com o negócio e suas regras. Portanto, para cada caso, esse grau de detalhamento pode variar — pode inclusive variar dentro do mesmo sistema. Há partes que, por algum motivo, precisam de mais detalhes, enquanto outras não.

Wiegers e Beatty (2013) sugerem que os requisitos sejam especificados com mais detalhes nas seguintes situações:

- o trabalho está sendo desenvolvido para um cliente externo;
- o desenvolvimento ou o teste serão terceirizados;
- os membros da equipe do projeto estão geograficamente distribuídos;
- os testes de sistema serão baseados nos requisitos;
- estimativas precisas são necessárias;
- a rastreabilidade dos requisitos é necessária.

Por outro lado, Wiegers e Beatty (2013) sugerem que os detalhes sejam evitados quando:

- o trabalho está sendo desenvolvido internamente para a sua empresa;
- os clientes estão extensivamente envolvidos;
- os desenvolvedores têm uma grande experiência no domínio;
- há experiência anterior, como quando uma aplicação está sendo desenvolvida para substituir outra;
- uma solução do tipo pacote será usada.



Saiba mais

Existem *softwares* de modelagem de diagramas UML que permitem não apenas o desenho do diagrama, mas também a inclusão da sua especificação, facilitando, posteriormente, a rastreabilidade. Caso você tenha acesso a uma dessas ferramentas, excelente, mas, se não tiver, sem problemas. Utilize uma ferramenta gratuita para fazer o diagrama e um editor de texto para realizar a especificação. Cada um desses artefatos cumprirá a sua finalidade: o desenho apoiará o diálogo com os diversos *stakeholders*, especialmente os usuários; e a especificação servirá para apoiar a equipe técnica do projeto. Só não esqueça de manter esses dois elementos sempre alinhados! Quando um deles for alterado, é importante checar se o outro não precisará também ser ajustado.

Existem diversos modelos para a especificação dos casos de uso, como os encontrados em Cockburn (2005), Dennis, Wixon e Roth (2012), Seidl *et al.,* (2012) e Guedes (2018). Cockburn (2005) tem preferência por um texto, sem formatação de tabela, mas com os títulos em destaque.

Aqui vamos utilizar uma versão derivada desses que é representada no Quadro 1. Cada empresa, no entanto, pode optar por um modelo customizado para as suas necessidades, incluindo mais ou menos detalhes. É importante apenas ressaltar que todos os elementos precisam ser úteis para alguém. Especificações desnecessariamente complicadas podem acabar entrando em desuso.

Quadro 1. Modelo de especificação de casos de uso

Identificador único do caso de uso	
Descrição	
Ator principal	
Atores secundários	
Pré-condições	
Gatilho	
Fluxo principal	
Ações do ator	**Ações do sistema**
Fluxo alternativo	
Ações do ator	**Ações do sistema**
Fluxo de exceção	
Ações do ator	**Ações do sistema**
Pós-condição	

Vamos conferir a seguir os elementos envolvidos neste *template* de especificação de casos de uso.

O **identificador único** se refere ao nome do caso de uso. Geralmente se utiliza um verbo no infinitivo, que caracteriza a ação que será realizada. Se você voltar na Figura 1, vai ver que naquele diagrama colocamos também uma sigla UC (advinda de *use case*), acompanhada de um número sequencial (UC1, UC2 etc.). Esse identificador reduzido não é obrigatório, mas pode ser útil para fazermos a rastreabilidade posteriormente.

A **descrição** visa a apresentar uma visão resumida do objetivo do caso de uso. Pode ser escrita em uma frase simples. Não é um lugar para especificar itens técnicos ou regras de negócio, serve apenas para descrever o objetivo geral do caso de uso.

Em seguida temos o **ator principal**, que é aquele que inicia o caso de uso, e os **atores secundários**, que são aqueles que participam na execução do caso de uso ou que fornecem informações para que o caso de uso possa ser realizado. Nem todo caso de uso terá atores secundários.



Fique atento

Vamos relembrar aqui o conceito de ator. De acordo com a UML (OBJECT MANAGEMENT GROUP, 2017, documento *on-line*), "Um ator especifica um papel que é desempenhado por um usuário ou qualquer outro sistema que interage com o sujeito".

Entende-se por sujeito o sistema que está sendo especificado. Alguns autores se referem ao sistema como SUD (*system under definition* ou *system under discussion*).

Um ator é um elemento externo ao sistema. Ele pode representar uma pessoa, um *hardware* especial ou um outro sistema. Chamamos de *hardware* especial aquele que é importante de ser destacado, como, por exemplo, algum sensor ou atuador que desempenhe função relevante para o sistema que está sendo modelado. Não precisamos representar a interação com *hardwares* triviais, como impressoras ou o próprio equipamento onde roda o *software*.

Quando uma funcionalidade é disparada pelo próprio sistema isso é representado usando um ator chamado Agendador de Tarefas (*scheduler*). Há quem represente o próprio sistema como sendo o ator que dispara esses casos de uso, mas isso vai contra a definição de ator (que é um elemento externo ao sistema).

Uma **pré-condição** é uma condição que tem que ser verdadeira para que o caso de uso tenha início. Ela normalmente é utilizada para estabelecer a sequência de casos de uso que realizam uma grande função de negócio (DENNIS; WIXON; ROTH, 2012). A pré-condição não é testada quando o caso de uso inicia, o caso de uso só inicia se ela for verdadeira. Por exemplo, se, para realizar uma compra, o usuário tem que estar logado, então o caso de uso que realiza o login já tem que ter sido executado com sucesso antes que o caso de uso de compra seja iniciado. Este é um dos casos mais comuns de pré-condição.

Uma **pós-condição** é o *status* em que o sistema deve ser deixado quando for concluído o caso de uso, ou seja, quando encerrar a execução do caso de uso o sistema garantirá que a pós-condição seja verdadeira. Quando a pós-condição for trivial, não é necessário documentá-la. Por exemplo, para o caso de uso Cadastrar Cliente, não é necessário colocar uma pós-condição "cliente deverá ter sido cadastrado com sucesso", pois é uma saída trivial de um caso de uso de cadastro.

Um **gatilho** é um evento que dá início ao caso de uso. Quando o caso de uso é chamado por um ator humano, então o gatilho é justamente a chamada feita por esse ator. Na prática, isso significa que será disponibilizado, por exemplo, um item de menu, um botão, um *link*, por meio do qual o usuário dá início à realização daquele caso de uso. Quando o disparo é automático, de acordo com determinadas circunstâncias, então o gatilho pode ser um determinado momento do tempo (por exemplo, 22:00h), ou pode ser um evento de controle (mudança do estado de algum objeto).

O **fluxo principal**, também chamado de fluxo básico, refere-se ao cenário mais comum de utilização do caso de uso, considerando que tudo deu certo. Ele também é conhecido como o "caminho feliz". O fluxo principal é geralmente mapeado como uma conversa entre o que o ator faz e o que o sistema responde, por isso as duas colunas no *template* apresentado no Quadro 1. Pode também ser usada uma única coluna, e cada parte do diálogo ser representada em uma linha. No fluxo principal se considera que todas as interações entre o ator e o sistema foram bem-sucedidas.



Fique atento

É necessário tomar cuidado para que o mapeamento do fluxo principal não caia no nível de detalhe de preenchimento de campos da interface. É preciso ter em mente que o caso de uso não é um pseudocódigo da implementação. Nunca esqueça de se perguntar se você não está especificando detalhes desnecessários.

Os **fluxos alternativos**, como o próprio nome sugere, são as possibilidades de cenários alternativos ao fluxo principal, conforme escolhas do usuário ou situações específicas do sistema. Não inclui fluxos para o tratamento de erro, mas sim escolhas que podem ser feitas pelo usuário.

Os **fluxos de exceção** demonstram como o sistema reagirá na presença de situações inesperadas. Os fluxos de exceção podem ocorrer dentro do fluxo principal ou dentro de um fluxo alternativo.



Fique atento

Não perca tempo especificando todos os detalhes de um caso de uso ou todos os casos de uso de um sistema, especialmente se eles não forem ser implementados em breve. Especificar casos de uso que só serão implementados em versões muito distantes é perda de tempo. Quando chegar o momento da implementação eles podem ter mudado ou simplesmente podem não ser mais necessários.

2 Especificação do fluxo principal

Conforme apresentado na seção anterior, o fluxo principal, também chamado de fluxo básico, refere-se ao cenário mais comum de utilização do caso de uso, considerando que tudo vai dar certo. A finalidade desse mapeamento é

identificar as interações que precisam ser previstas por parte do usuário e como o sistema vai tratar essas entradas. Entretanto, é importante manter certo distanciamento dos detalhes da interface, sob pena de ficarmos presos somente a cores, campos e botões. Especificar o fluxo básico é mais do que especificar interfaces; é expressar um caso de sucesso que entrega valor ao usuário!

Agora vamos ver um exemplo de uma especificação relacionada ao diagrama apresentado na Figura 1. Vamos considerar o caso de uso UC3 – Realizar Compra. O primeiro passo é compreender o que está desenhado no diagrama. Imediatamente identificamos o símbolo do ator principal, neste caso, um Comprador Logado. Ele indica que existe uma **pré-condição**, que é o *login* já ter sido realizado. Não foram identificados atores secundários no diagrama, e este é o momento de se perguntar se existe algum que foi esquecido. A resposta neste exemplo é "não", mas, se houver, o digrama de casos de uso deve ser atualizado, para refletir essa nova descoberta. Neste exemplo, não existe **pós-condição**.

O **gatilho** para que este caso de uso tenha início é a ação do usuário, solicitando a sua execução. Isso é identificado no diagrama de caso de uso pela associação, por meio de uma linha contínua, que liga o ator Comprador Logado ao caso de uso. Se fosse uma tarefa disparada automaticamente pelo sistema, o ator seria um agendador de tarefas e o gatilho seria a condição de guarda desse agendador (por exemplo, a chegada de um momento específico no tempo).

O próximo passo é verificar se existe algum tipo de relacionamento com outros casos de uso. No diagrama existe um relacionamento de *include* com o caso de uso UC2 – Pesquisar Produtos e outro relacionamento de *include* com o caso de uso UC4 – Realizar Pagamento. Isso significa que esses dois casos de uso deverão ser iniciados dentro do caso de uso que está sendo descrito.

A partir dessas informações que foram provenientes do diagrama de casos de uso, é preciso determinar como ocorrerá o diálogo entre o ator e o sistema, para realizar o objetivo do caso de uso. Essa conversa é descrita nos passos do **fluxo principal**, que são numerados sequencialmente e ordenados na coluna Ações do Ator e na coluna Ações do Sistema.

Acompanhe no Quadro 2 como ficou a especificação deste fluxo principal.

Quadro 2. Especificação do caso de uso UC3 – Realizar Compra – Fluxo Principal

Identificador único do caso de uso	UC3 – Realizar Compra
Descrição	Permite ao comprador realizar a compra de um produto
Ator principal	Comprador Logado
Atores secundários	×
Pré-condições	O caso de uso UC1 – Realizar Login ter sido executado com sucesso e o comprador estar logado
Gatilho	Comprador Logado seleciona Realizar Compra
Fluxo principal	
Ações do ator	**Ações do sistema**
	1 – Sistema exibe categorias de produtos
2 – Comprador Logado seleciona categoria de produto	
	3 – Sistema executa UC2 – Pesquisar Produtos
4 – Comprador Logado seleciona produto	
	5 – Sistema solicita quantidade
6 – Comprador Logado informa a quantidade	
	7 – Sistema exibe opções de entrega
8 – Comprador Logado seleciona opção de entrega	
	9 – Sistema executa UC4 – Realizar Pagamento
Pós-condição	Não se aplica

3 Especificação dos fluxos alternativos e de exceção

Conforme definido na primeira seção deste capítulo, os fluxos alternativos são as possibilidades de cenários alternativos ao fluxo principal, conforme escolhas do usuário ou situações específicas do sistema. Não consiste em fluxos para o tratamento de erro, mas sim em escolhas que podem ser feitas pelo usuário, para executar um cenário alternativo de sucesso.

Uma vez mapeado o fluxo principal, vem a pergunta: existe alguma coisa que possa mudar o fluxo principal? O usuário tem alternativas para algum dos passos? Essas alternativas são cenários diferentes e resultam em fluxos alternativos.

Suponha que o Comprador Logado possa adicionar um novo endereço de entrega quando estiver selecionando a opção de entrega. Observe no Quadro 3 que é incluído um fluxo alternativo A8 que permitirá realizar essa tarefa. Este fluxo é nomeado com a letra A (alternativo), seguido do número 8, indicando que é a partir do passo 8 do fluxo básico que este fluxo é acionado. No fluxo principal, este fluxo alternativo A8 também é registrado no passo 8. A forma de numerar pode ser diferente desta, à escolha da empresa, desde que forneça interpretação similar e que seja utilizada de forma consistente em todas as especificações.

Caso a alteração de endereço fosse realizada por um outro caso de uso, então o diagrama teria que ser alterado para inserir esse caso de uso novo e também o seu relacionamento de *extend* com o caso de uso em questão. Na especificação seria chamado este novo caso de uso.

Os fluxos de exceção demonstram como o sistema reagirá na presença de situações inesperadas, ou seja, ele mapeia caminhos de insucesso. Os fluxos de exceção podem ocorrer tanto dentro do fluxo principal quanto dentro de um fluxo alternativo.

Para cada passo do fluxo principal deve-se fazer a pergunta: o que pode dar errado? Cada resposta a essa pergunta vai determinar um fluxo de exceção. Neste exemplo, foram identificadas duas situações: uma em que o produto não está disponível (isso ocorre no passo 4 do fluxo básico) e outra em que a quantidade não está disponível (isso ocorre no passo 6 do fluxo básico). O tratamento para cada uma das exceções será planejado na seção de fluxo de exceção correspondente a cada possibilidade.

Assim como fizemos no fluxo principal, precisamos nos questionar o que pode dar errado no fluxo alternativo. Neste exemplo foi identificado que o Comprador Logado pode ter digitado um CEP inválido (passo A7.2). Isso também é mapeado como um fluxo de exceção, que sai de dentro do fluxo alternativo.

Quadro 3. Especificação do caso de uso UC3 – Realizar Compra – Fluxo Alternativo

Identificador único do caso de uso	UC3 – Realizar Compra
Descrição	Permite ao comprador realizar a compra de um produto
Ator principal	Comprador Logado
Atores secundários	×
Pré-condições	O caso de uso UC1 – Realizar Login ter sido executado com sucesso e o comprador estar logado
Fluxo principal	
Ações do ator	**Ações do sistema**
	1 – Sistema exibe categorias de produtos
2 – Comprador Logado seleciona categoria de produto	
	3 – Sistema executa UC2 – Pesquisar Produtos
4 – Comprador Logado seleciona produto	
	5 – Sistema solicita quantidade
6 – Comprador Logado informa a quantidade	
	7 – Sistema exibe opções de entrega
8 – Comprador Logado seleciona opção de entrega (A8)	
	9 – Sistema executa UC4 – Realizar Pagamento
Fluxo Alternativo A8 – Alterar endereço de entrega	
A8.1 – Comprador Logado seleciona Alterar Endereço	
	A8.2 – Sistema solicita novo endereço
A8.3- Comprador Logado preenche novo endereço	
	A8.4 – Sistema salva novo endereço
Pós-condição	Não se aplica

O Quadro 4 ilustra como a especificação do caso de uso ficará a partir dessas novas descobertas.

Quadro 4. Especificação do caso de uso UC3 – Realizar Compra – Fluxos de Exceção

Identificador único do caso de uso	UC3 – Realizar Compra
Descrição	Permite ao comprador realizar a compra de um produto
Ator principal	Comprador Logado
Atores secundários	×
Pré-condições	O caso de uso UC1 – Realizar Login ter sido executado com sucesso e o comprador estar logado
Fluxo principal	
Ações do ator	**Ações do sistema**
	1 – Sistema exibe categorias de produtos
2 – Comprador Logado seleciona categoria de produto	
	3 – Sistema executa UC2 – Pesquisar Produtos
4 – Comprador Logado seleciona produto (E4)	
	5 – Sistema solicita quantidade
6 – Comprador Logado informa a quantidade (E6)	
	7 – Sistema exibe opções de entrega
8 – Comprador Logado seleciona opção de entrega (A8)	
	9 – Sistema executa UC4 – Realizar Pagamento

(Continua)

(Continuação)

Quadro 4. Especificação do caso de uso UC3 – Realizar Compra – Fluxos de Exceção

Fluxo Alternativo A8 – Alterar endereço de entrega	
A8.1 – Comprador Logado seleciona Alterar Endereço	
	A8.2 – Sistema solicita novo endereço
A8.3- Comprador Logado preenche novo endereço (E.A8.3)	
	A8.4 – Sistema salva novo endereço
Fluxo de Exceção E4 – Produto indisponível	
	E4.1 - Sistema exibe mensagem "Produto indisponível no momento. Deseja voltar para seleção de produtos?"
E4.2 - Comprador Logado confirma que quer voltar para seleção de produtos	
	E4.3 - Sistema retorna para a seleção de produtos
Fluxo de Exceção E6 – Quantidade indisponível	
	E6.1 - Sistema exibe mensagem "Quantidade indisponível. Quer reduzir a quantidade?"
E6.2 - Comprador Logado confirma que quer reduzir a quantidade	
	E5.3 - Sistema retorna para a seleção da quantidade
Fluxo de Exceção E.A8.3 – CEP inválido	
	E.A8.3.1 - Sistema exibe mensagem "CEP inválido. Corrigir CEP."
	E.A8.3.2 - Sistema retorna para a entrada do CEP
Pós-condição	Não se aplica

Tratamento de casos de uso CRUD

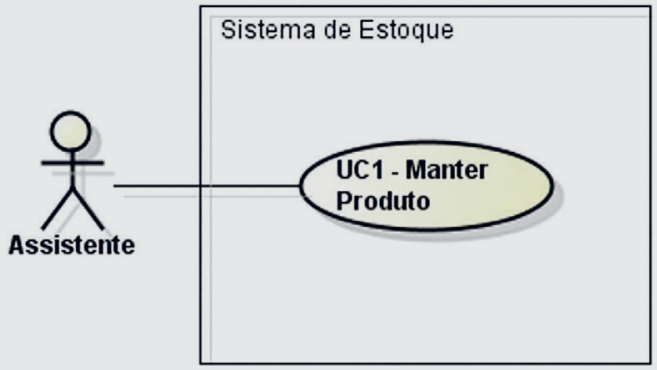
Uma pergunta que sempre surge quando falamos na abordagem orientada a casos de uso é como tratar as questões relacionadas aos casos de uso que manipulam cadastros, ou seja, as operações CRUD (*create*, *retrieve*, *update* e *delete*). A resposta é: não existe uma forma padrão! Há os que defendem que essas operações estejam agrupadas em um caso de uso mais genérico, denominado Manter ou Gerenciar, em vez de existir um caso de uso para cada uma das operações. O principal argumento a favor dessa abordagem é que tanto o diagrama quanto as especificações ficam mais simples. Essa é a forma normalmente adotada na prática.

Neste caso, a especificação tratará como fluxo principal o fluxo mais frequente, e como fluxos alternativos, os demais fluxos.



Exemplo

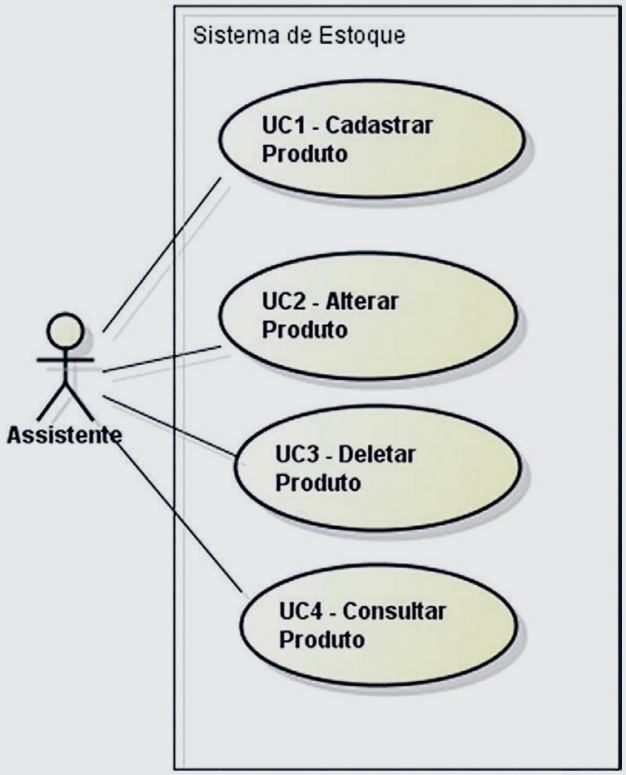
O diagrama de casos de uso a seguir ilustra a representação das operações CRUD como um único caso de uso, Manter Produto.



Há, por outro lado, os que defendem que as operações devem ser tratadas de forma distinta, uma vez que podem, inclusive, ter níveis de permissão de acesso diferentes e serem realizadas por pessoas diferentes (COCKBURN, 2005). Quando isso ocorrer, é importante ter em mente que deverá existir uma boa justificativa, ou será gasto um esforço adicional desnecessário em especificações separadas.

Exemplo

O diagrama de casos de uso a seguir ilustra a representação das operações CRUD como casos de uso separados: UC1 – Cadastrar Produto, UC2 – Alterar Produto, UC3 – Deletar Produto e UC4 – Consultar Produto. Imagine que este sistema tenha dezenas de cadastros e todos eles com as quatro operações. O diagrama de casos de uso ficaria extremamente poluído, dificultando a identificação dos casos de uso mais relevantes.



Os diagramas e as especificações de caso de uso são ferramentas complementares que podem ser utilizadas em uma variedade grande de situações. Eles se aplicam a pequenos projetos e a projetos de grande porte. Eles são especialmente úteis em projetos nos quais existe uma interação expressiva entre os usuários e o sistema.



Fique atento

Se você ainda tiver dúvidas sobre a relevância das especificações de caso de uso, veja o que diz Alistair Cokburn (2005, p. 32), um dos precursores dos métodos ágeis: "Os casos de uso são amplamente populares porque eles contam histórias coerentes de como o sistema irá se comportar em uso. Os usuários do sistema conseguem ver o que o sistema será.".

O caso de uso agrega valor em dois momentos (COCKBURN, 2005):

- o primeiro, quando os casos de uso são nomeados como objetivos do usuário e compõem uma lista que revela o escopo do sistema e o seu propósito, apoiando atividades de estimativa e priorização;
- o segundo, quando os autores dos casos de uso precisam questionar o que pode dar errado no fluxo básico, identificando como o sistema deve reagir. Neste caso, podem ser descobertas coisas surpreendentes que nem a equipe e nem o cliente tinham pensado anteriormente.



Referências

COCKBURN, A. *Escrevendo casos de uso eficazes*: um guia prático para desenvolvedores de software. Porto Alegre: Bookman, 2005.

DENNIS, A.; WIXON, B.; ROTH, R. *System analysis and design*. 5. ed. Hoboken: John Wiley and Sons, 2012.

GUEDES, G. *UML 2*: uma abordagem prática. 3. ed. São Paulo: Novatec, 2018.

OBJECT MANAGEMENT GROUP. *Unified Modeling Language®*: OMG UML® v2.5.1. [*S. l.*], 2017. Disponível em: https://www.omg.org/spec/UML/About-UML/. Acesso em: 13 mar. 2020.

SEIDL, M. *et al*. *UML@Classroom:* an introduction to object-oriented modeling. Cham: Springer, 2012.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Parte 4

Especificação de Requisitos Funcionais Utilizando Histórias de Usuário

O conteúdo deste livro é disponibilizado por SAGAH.

SAGAH SOLUÇÕES EDUCACIONAIS INTEGRADAS

Especificação de requisitos funcionais utilizando histórias de usuário

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Reconhecer os elementos de um cenário de desenvolvimento ágil.
- Identificar os elementos de uma história de usuário.
- Aplicar histórias de usuário para especificar requisitos funcionais.

Introdução

A identificação e o registro dos requisitos de *software* constituem o primeiro, e o mais fundamental, passo para o desenvolvimento de um bom produto de *software*. Sua importância é reconhecida tanto por aqueles que praticam processos ágeis, como por aqueles que utilizam métodos mais tradicionais de desenvolvimento. As consequências de requisitos mal definidos são sentidas, principalmente, na forma de retrabalho, custos mais elevados de desenvolvimento e insatisfação do usuário.

Independentemente da forma que se escolha para realizar as atividades relacionadas a requisitos, um ponto é chave: o foco na entrega de valor para o cliente. A forma como esses requisitos serão registrados e gerenciados vai depender das diversas variáveis do contexto no qual o projeto está inserido, mas o foco deve ser sempre na entrega de valor. Esse deveria ser o mantra repetido e internalizado por todas as equipes de desenvolvimento do mundo, independentemente do processo de trabalho que elas adotam. Se não houver valor na entrega, ela nem deveria ser realizada.

Uma forma pela qual as equipes ágeis vêm trabalhando com os requisitos é o uso de histórias de usuário. Neste capítulo você vai estudar o ambiente ágil de desenvolvimento de *software,* vendo como identificar os elementos que compõem uma história de usuário e como aplicar as histórias de usuário.

1 Desenvolvimento ágil de *software*

Métodos ágeis vêm se consolidando como uma das formas mais utilizadas no desenvolvimento de *software*. Embora a sua origem anteceda os anos 2000, esses métodos ganharam mais força recentemente, especialmente devido à pressão do mercado por entregas mais rápidas e frequentes. Esse é o caso, por exemplo, do mercado de aplicativos para *smartphones*. A Apple Store tem mais de 100 milhões de usuários ativos que consomem mais de 2 milhões de aplicativos.

Os princípios de agilidade começaram a ser fortemente difundidos a partir do Manifesto para o Desenvolvimento Ágil de *Software*, ou, simplesmente Manifesto Ágil (BECK *et al*., 2001). Esse documento foi produzido pelo esforço conjunto de diversos profissionais de *software*, que se reuniram em 2001 para discutir e organizar um conjunto de valores e princípios que passaram a ser considerados como a base do desenvolvimento ágil. Entre os profissionais estavam Alistair Cockburn (criador do método *Crystal Clear* e defensor da escrita de casos de uso eficazes); Kent Beck (criador do método XP — *eXtreme Programming*); e, Ken Schwaber e Jeff Sutherland (criadores do Scrum).

Apesar de divergirem entre si em alguns aspectos, e em alguns casos serem até mesmo concorrentes no mercado, esses profissionais chegaram à conclusão que partilhavam dos mesmos valores e princípios e chegaram a um consenso que está expresso no próprio documento gerado (BECK *et al*., 2001).

Os valores ágeis, formalizados no Manifesto Ágil em 2001, estão representados na Figura 1. Como se pode observar, o foco está mais sobre as pessoas e a comunicação com o cliente do que sobre o formalismo de processos, planos e contratos. Porém, isso não quer dizer que esses metodologistas eram contra processos; na verdade, eles eram a favor de processos que agregassem valor ao produto final entregue ao usuário, e não processos que agregassem burocracias e documentações desnecessárias.

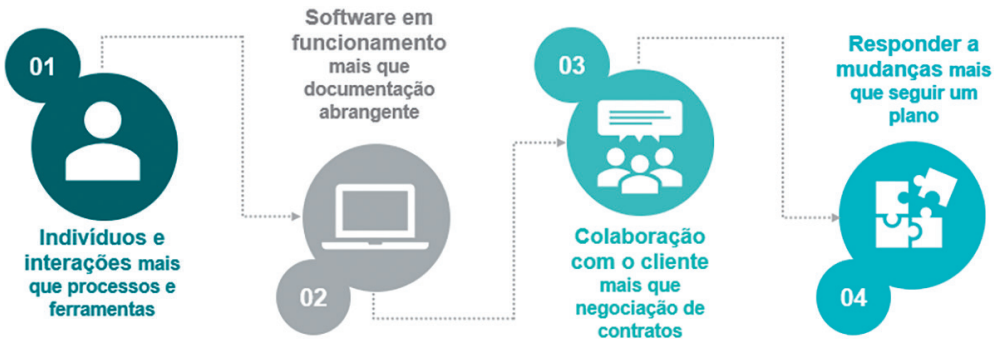


Figura 1. Valores dos métodos ágeis.

Fonte: Adaptada de Beck *et al.* (2001).

Os valores ajudam a construir o *mindset* ágil da equipe para produzir *softwares* melhores. Esse *mindset* se refere às atitudes que o time deve ter diante das tarefas que ele precisa desempenhar (STELLMAN; GREEN, 2017). No entanto, no dia a dia, nem sempre é tão fácil perceber como esses valores estão presentes. Para isso, o Manifesto Ágil definiu um conjunto de 12 princípios ágeis que ajudam a tornar mais palpáveis esses valores nas tarefas do dia a dia.

Saiba mais

Se quiser conhecer um pouco mais sobre o Manifesto para o Desenvolvimento Ágil de *Software,* você pode pesquisar a versão original do manifesto na *web* — é bem fácil de encontrar. No *site* oficial do manifesto, é possível conhecer sua história, valores e os 12 princípios ágeis, bem como a biografia dos profissionais que participaram desse encontro. Esses continuam sendo os princípios norteadores de todas as metodologias ágeis atualmente em vigor.

O método SCRUM

Para compreender melhor os ambientes ágeis, optamos por utilizar como exemplo o método SCRUM, uma vez que ele é considerado hoje o método mais utilizado no mundo ágil.

O SCRUM é um *framework* para desenvolver, manter e entregar produtos complexos (SCHWABER; SUTHERLAND, 2017). A Figura 2 representa o ciclo de desenvolvimento utilizado no SCRUM. Ela nos será útil para explicar as características desses ambientes.

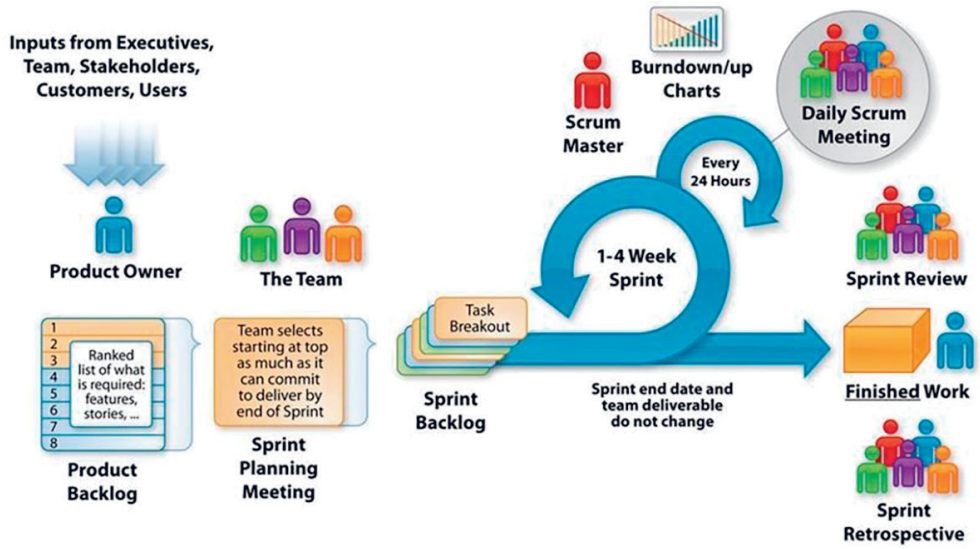


Figura 2. O ciclo de desenvolvimento SCRUM.

Fonte: Neon Rain Interactive (2020, documento *on-line*).

O princípio básico do SCRUM é de que o desenvolvimento de *software* deve ser dividido em entregas menores, de uma a quatro semanas, denominadas *sprints*. O desenvolvimento é todo organizado em torno das *sprints*. Ao final de uma *sprint*, é gerado um item potencialmente entregável para o cliente.

O *backlog* é o repositório de todas as requisições dos clientes e é gerenciado pelo *Product Owner* (PO, ou dono do produto). É o PO quem prioriza o conteúdo do *backlog* que será alocado à *sprint*. Só pode ser alocado à *sprint* um item do *backlog* que já esteja em um nível de detalhe suficiente para que a equipe possa implementar. Um item de *backlog* geralmente é uma história de usuário, mas pode ser qualquer solicitação do cliente.

Quem planeja a *sprint* é a equipe, estimando os itens do *backlog* e discutindo suas dificuldades para implementar. A ideia é que a data final da *sprint*, uma vez definida, não seja alterada, para que o cliente possa receber a entrega na data originalmente planejada. Ninguém tem autoridade de alterar a *sprint*, apenas a equipe de desenvolvimento. Isso previne a inserção de itens não previstos no decorrer da *sprint*.

O SCRUM também prevê alguns eventos, denominados **cerimônias**, que são reuniões com propósito específico e duração máxima estipulada, e são listadas a seguir:

- ***Sprint Planning Meeting* (Reunião de Planejamento da *Sprint*):** planejamento de como será realizada a *sprint*. É realizada pelo time SCRUM e seu esforço não deve ultrapassar 8 horas para uma *sprint* de um mês.
- ***Daily SCRUM Meeting* (Reunião Diária do SCRUM):** constitui a principal ferramenta de acompanhamento e comunicação. São reuniões curtas que acontecem todos os dias para que a equipe avalie o que fez, o que fará e o que não está andando. No XP elas são chamadas de *standup meetings* (literalmente reuniões em pé).
- ***Sprint Review* (Revisão da *Sprint*):** visa a reunir equipe e os principais *stakeholders* para avaliar o que estava planejado, o que foi realizado e quais as perspectivas para as próximas *sprints*. Tem a duração máxima de 4 horas para uma *sprint* de um mês.
- ***Sprint Retrospective* (Retrospectiva da *Sprint*):** o objetivo é que o time SCRUM faça uma avaliação interna de como as coisas ocorreram dentro da *sprint*. A partir daí são propostas as melhorias para as próximas *sprints*. Dura no máximo 3 horas para uma *sprint* de um mês.

O SCRUM define um papel central relacionado aos requisitos, que é a figura do ***Product Owner***. O PO é uma pessoa, e não um grupo de pessoas, mas ele pode até representar um grupo de pessoas ou um comitê. Sua função é gerenciar o *backlog* do produto. Outro papel é o **SCRUM Master**, cuja atribuição é a remoção dos impedimentos que atrapalham a realização da *sprint*. Ele não é um gerente de projetos, pois a ideia é que a equipe seja autogerenciada e autônoma. Ele é também um membro da equipe que realiza atividades técnicas. O papel do **time SCRUM** é ocupado por desenvolvedores e testadores.

Os requisitos geralmente são expressos no formato de histórias do usuário, que são os itens que compõem o *backlog* do produto. As histórias de usuário, uma vez compreendidas e detalhadas, serão a base para as estimativas que a equipe irá realizar para compor o seu planejamento da *sprint*. O *backlog* da *sprint* contém os itens que foram acordados para a *sprint*, além do plano para o seu desenvolvimento. Vamos conversar mais sobre histórias do usuário na próxima seção.

O que caracteriza um ambiente ágil é a presença de características comuns, que são o foco no cliente e no que agrega valor para esse cliente, a entrega frequente de valor para esse cliente (entregas mais curtas e rápidas), a comunicação aberta e transparente tanto dentro time de desenvolvimento quanto com o cliente.

O caso das *startups* de *software*

As *startups* de *software*, ou simplesmente *startups*, convivem em um ambiente dinâmico, caótico e coberto de incertezas, vivendo sob a constante pressão do mercado e dos concorrentes, ao mesmo tempo em que precisam lidar com a permanente escassez de recursos. Um projeto falho pode colocar sua existência em risco, e a maioria delas deixa de existir em um prazo de dois anos (GIARDINO *et al.*, 2016).

Diversos são os fatores que contribuem para a falha das *startups* em sua fase inicial. A maioria desses fatores é de natureza interna, muito mais do que de natureza externa, como desenvolvedores inteligentes, mas inexperientes, negligenciando aspectos importantes de engenharia de *software*; o produto muitas vezes não é apenas um produto, mas uma família de produtos, levando a custos mais elevados de desenvolvimento; o produto não tem um dono, o que dificulta a tomada de decisão sobre as funcionalidades; não existe um planejamento estratégico para o desenvolvimento do produto; as tecnologias envolvidas podem não suportar a evolução do produto etc. (CROWNE, 2002).

Outros fatores surgem posteriormente, quando o produto já está no mercado e a *startup* enfrenta dificuldades para crescer: conflitos entre os membros que estavam desde o início e os novos membros; o *software* se torna complexo e instável; torna-se difícil gerenciar os requisitos; as expectativas sobre o produto se tornam muito altas, entre outros (CROWNE, 2002).

Devido às características das *startups*, focadas em entregas rápidas e falhas igualmente rápidas, o processo de gerenciamento do ciclo de vida mais utilizado é o processo ágil e suas variantes *lean* (GIARDINO *et al.*, 2014). Mesmo assim, em geral elas não seguem rigorosamente processo algum, preferindo prototipar rapidamente de modo a testar um produto mínimo viável (MVP). Qualidade de *software* não é uma de suas grandes prioridades (GIARDINO *et al.*, 2014) e as consequências aparecem posteriormente, quando os problemas no desenvolvimento gerados pela ausência de engenharia podem chegar a inviabilizar a evolução do produto e o crescimento da empresa (BERG *et al.*, 2018), ou, até mesmo, levá-la ao fracasso. Processos com menos cerimônia e mais reativos funcionam no início, mas tendem a ser insuficientes à medida que o produto ganha mercado e necessita de robustez, escalabilidade, *performance* e foco no consumo de energia (BERG *et al.*, 2018).

Não queremos, contudo, dizer que se deva adotar processos burocráticos e cheios de cerimônia, mas devemos, sim, buscar a implementação da filosofia ágil, mesclada com a visão da perenidade da organização.

O que se percebe é que aspectos relacionados à engenharia de *software*, em geral, são negligenciados pelas *startups* no início do seu desenvolvimento, com destaque para os processos relacionados a requisitos, uma vez que o foco dessa fase inicial é ganhar rapidamente o mercado. Os requisitos acabam sendo os mais negligenciados, o que é um aspecto contraditório, uma vez que se ganha o mercado com um produto que atende justamente aos anseios desse mercado, ou seja, que atende aos requisitos.

Ao crescer e ganhar em escala, a *startup* sente os efeitos negativos da falta de engenharia de *software* do início e então precisa se voltar para dentro e organizar a casa. O primeiro passo geralmente é começar a implementar métodos ágeis com mais seriedade, focando aspectos antes negligenciados, como os requisitos, por exemplo. Nesses casos, é comum a utilização do SCRUM e das histórias de usuário.



Saiba mais

Para conhecer mais sobre como as *startups* têm lidado com as questões de engenharia de *software*, leia o artigo de Giardino *et al.* (2016). Nele os autores mapeiam o estado das *startups* em um modelo denominado *Greenfield Startup Model*, apontando as lacunas que existem na sua criação e seus efeitos sobre a *startup*.

2 Identificação dos elementos da história do usuário

Quando os métodos ágeis surgiram, no final da década de 1990, trouxeram uma forma diferente de pensar tanto a organização do time de desenvolvimento quanto a organização dos requisitos, que passaram a ser tratados como histórias de usuário. O conceito de história de usuário foi criado por Kent Beck, na metodologia ágil XP (*eXtreme Programming*).



Saiba mais

A definição original de história do usuário, dada por Kent Beck e Martin Fowler é: "A história é a unidade de funcionalidade em um projeto XP. Nós demonstramos o progresso entregando código integrado e testado, que implementa uma história. Uma história deveria ser compreendida pelos clientes, testável pelo desenvolvedor, de valor para o cliente e pequena o suficiente para que os programadores possam construir uma meia dúzia delas em uma iteração."

Fonte: Leffingwell (2011).

De acordo com Patton (2014), "Histórias foram assim nomeadas devido a como é esperado que elas sejam usadas, não devido à forma como você está tentando escrevê-las". Em essência, uma história do usuário é um cenário de uso de um produto por um tipo específico de usuário. Mais importante do que a história registrada em si é a comunicação com o cliente para chegar a um entendimento comum. No caso do XP, a história é escrita pelo próprio cliente. No caso do SCRUM isso é feito pelo PO, que é quem representa os interesses dos clientes e usuários.

Roy Jeffrey, um dos criadores do XP, estabeleceu os 3 Cs da história de usuário: **C**artão, **C**onversa e **C**onfirmação. Vamos ver cada um deles a seguir.

Cartão (*Card*)

Com o passar do tempo e o uso nas organizações, as histórias de usuário passaram a ter um formato mais padronizado, com afirmações escritas em primeira pessoa, contendo o papel (quem), a ação (o que) e o resultado da ação ou o valor de negócio da história (porque), conforme ilustrado na Figura 3.

Essa forma é chamada de **expressão da história do usuário** ou **voz do usuário** (LEFFINGWELL, 2011). Alguns denominam também de **cartão (*card*) da história**, uma alusão à sua escrita simples em cartões ou notas adesivas tipo Post-it®. Embora esses meios físicos possam ser usados, também é comum o uso de ferramentas de *software* para essa finalidade.

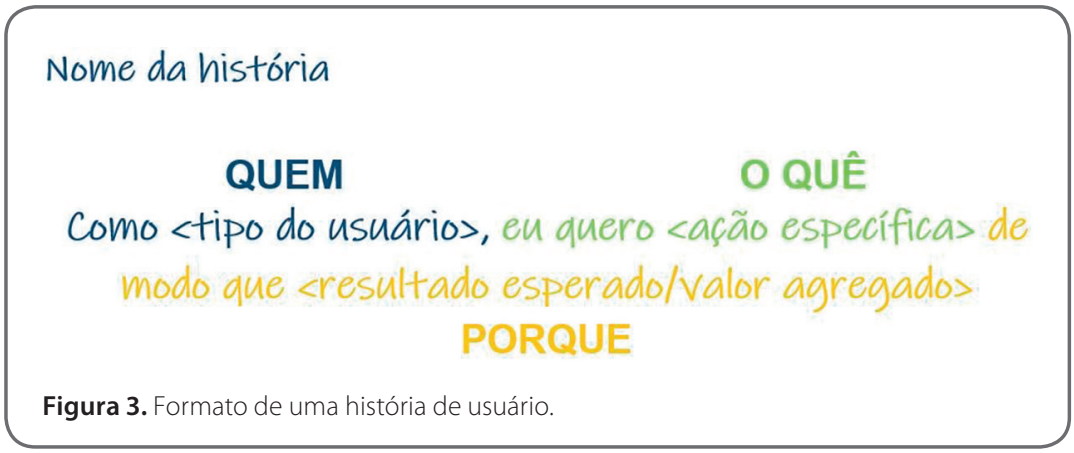


Figura 3. Formato de uma história de usuário.

Podemos entender que o **papel** (QUEM) representa quem se beneficiará da funcionalidade, e não quem está solicitando a funcionalidade, uma vez que podem ser *stakeholders* diferentes. A **atividade** (O QUÊ) descreve o que será a funcionalidade (ou seja, podemos entender como sendo o requisito funcional). O **resultado** (PORQUE) ou valor agregado, traduz a perspectiva da utilidade da funcionalidade no contexto de utilização, ou seja, qual é o seu valor de negócio. Representamos, portanto, três elementos: quem (papel), o quê (atividade), para quê (valor).



Exemplo

Vamos acompanhar o exemplo de um sistema de *check-in* de uma companhia aérea:

- *Como Passageiro, eu quero fazer meu check-in via celular, para não perder tempo na fila do aeroporto.*
- *Como Passageiro VIP, eu quero poder escolher assentos preferenciais, para viajar mais confortável.*
- *Como Atendente do Check-in, eu quero emitir etiquetas de bagagem, para despachar as malas dos passageiros.*
- *Como Atendente do Embarque, eu quero ler os cartões de embarque, para liberar o acesso dos passageiros ao avião*

Precisamos ficar atentos sobre os elementos da história de usuário. Você realmente deve se preocupar com o **quem**, ou seja, identifique quem serão os usuários e os clientes (pois, às vezes, não são a mesma pessoa). Não use o termo genérico usuário. Pense nos diversos perfis que podem ser usuários do produto. Se achar conveniente, faça um mapeamento de personas, para poder identificar os diversos perfis que irão interagir com o *software* (PATTON, 2014).

Patton ainda fala sobre o que realmente deve ser tratado na porção **o quê** da história. Ele recomenda que se fale sobre o que o *software* deve fazer, mesmo aquilo que aparentemente o usuário não percebe, como, por exemplo, obter a autorização da operadora do cartão de crédito para que uma compra seja realizada.

Finalmente, Patton (2014) nos diz que é importante falar sobre o **porquê**, pois existem muitas coisas escondidas atrás dos porquês. Continue escavando para entender por que é importante para o usuário, para a gerência do usuário, para o negócio.

O cartão pode ser compreendido como uma espécie de *token* que sumariza uma necessidade, mas os detalhes ainda serão definidos. Não podemos assumir que apenas o cartão irá conter tudo o que é necessário saber sobre a história para que ela possa ser implementada (LEFFINGWELL, 2011). Os detalhes virão na (C)onversa e nos (C)ritérios de aceitação.

É a partir das histórias do usuário que as tarefas serão derivadas, como, por exemplo, codificar a história, testar a história, documentar o *help* do usuário e assim por diante.

Conversa (*Conversation*)

Se voltarmos aos valores que embasam os métodos ágeis, vamos nos lembrar que um deles nos diz que devemos nos preocupar mais com a colaboração com o cliente do que com a negociação de contratos. Isso nos remete ao princípio fundamental da engenharia de requisitos: a comunicação. Nada substitui uma boa conversa (*conversation*) para que os requisitos sejam discutidos, interpretados, compreendidos. O objetivo da história de usuário é ser, justamente, o ponto de partida dessa conversa.

O que se espera dos *stakeholders* em ambientes ágeis é o seguinte (LEFFINGWEL, 2011):

- As decisões precisam ser tomadas pelos *stakeholders* no momento adequado em que são necessárias, especialmente as referentes ao escopo e às prioridades.
- A participação ativa dos *stakeholders* é necessária.
- As equipes precisam ter uma visão organizacional e enxergar a necessidade de integração com outros sistemas.
- Profissionais de produção e suporte precisam ser envolvidos desde o início.
- Profissionais que atuarão na manutenção do produto (caso seja um time diferente do time de desenvolvimento) deverão ser envolvidos também no início.

Aqui vale uma ressalva quanto aos que usam SCRUM. No SCRUM, o papel do PO é atuar na definição dos itens do *backlog,* bem como na sua priorização, mas isto não quer dizer que ele deva ser a única fonte de informação para a equipe de desenvolvimentos. Ele representa a voz de todos os demais *stakeholders*, e deve ser garantido que assim o seja.

Confirmação ou Critérios de aceitação (*Confirmation* ou *Acceptance Criteria*)

O "C" da confirmação (*confirmation*) se refere aos critérios de aceitação, ou seja, critérios complementares que ajudam a esclarecer como será validada a história. Podemos entender que são uma espécie de complemento da especificação da história, uma vez que a simplicidade do cartão não é suficiente para registrar tudo o que uma história significa. Alguns defendem, inclusive, que eles sejam escritos no verso do cartão da história, assumindo, claro, que um cartão real esteja sendo usado.

Critérios de aceitação não devem ser confundidos com testes funcionais ou testes unitários. Os casos de teste em si serão mais aprofundados e irão prever os vários tipos de caminhos dentro de um código. Alguns chamam esses critérios de testes de aceitação da história (LEFFINGWEL, 2011).

Os critérios de aceitação podem expressar fluxos alternativos que surgem ao longo da conversa com o usuário, bem como limites que estabelecem alguma fronteira.



Exemplo

Vamos acompanhar o exemplo da seguinte história de usuário de um sistema de *check-in* de uma companhia aérea:

- *Como Passageiro, eu quero fazer meu check-in via celular, para não perder tempo na fila do aeroporto.*

Os critérios de aceitação seriam:

- *Apresentar os assentos para seleção, destacando os assentos vazios*
- *Destacar os assentos especiais, permitindo compra na hora do check-in*
- *Apresentar a possibilidade de compra de bagagem extra no momento do check-in*

Boas histórias de usuário

Grande parte do esforço da equipe de desenvolvimento é empregado em compreender as histórias de usuário e em escrever critérios de aceitação para que elas possam ser testadas posteriormente. Algumas dicas para realizar adequadamente essa atividade foram projetadas por Bill Wake, que criou o acrônimo em inglês INVEST (LEFFINGWELL, 2011), conforme ilustra a Figura 4. Vamos ver a explicação de cada um desses itens na sequência.



Figura 4. Dicas para uma boa história de usuário.

Fonte: Adaptada de Leffingwell (2011).

Uma história de usuário deve ser **independente (*independent*)**, ou seja, ela pode ser identificada, discutida, implementada e até mesmo liberada para uso de forma independente de outras histórias. Este é um dos itens mais complexos, pois envolve o nível de granularidade da história de usuário. Se a história precisa ser independente, é preciso que seja possível liberá-la para uso também de forma independente dentro do prazo de uma *sprint*.



Saiba mais

Quando uma história do usuário é muito grande, ela é chamada de **épico**. Um épico pode ser dividido em duas ou mais histórias (COHN, 2004). Não há problema em ter um épico no *backlog* de produto. No entanto, para ser priorizado para fazer parte de uma *sprint*, ele deverá ser particionado em histórias independentes.

Toda história do usuário deve ser **negociável (*negotiable*)**. Isso quer dizer que ela não é um contrato imutável. O usuário pode mudar de ideia e a história poderá ser adaptada. Ela é mais uma forma de melhorar a comunicação entre o negócio e a equipe de desenvolvimento.

Um dos princípios básicos que permeiam os métodos ágeis é a entrega do maior valor possível para o cliente, dentro das limitações de tempo da *sprint*. Isso quer dizer que cada história deve ser **valiosa (*valuable*)**, ou seja, ela deve agregar valor para alguém. Se ela não agrega valor, ela não deveria estar alocada à *sprint*.

Uma história precisa ser **estimável (*estimable*)**. Ser estimável quer dizer que é possível estabelecer uma estimativa da complexidade da história para, no mínimo, saber se ela é compatível com a duração da *sprint* ou se precisa ser dividida. Não é nosso objetivo neste capítulo detalhar procedimentos de estimativa. O que importa para garantir que temos uma boa história é que a sua complexidade possa ser estimada e se possa alocá-la com confiança à *sprint*. Ao discutir com o time a complexidade de uma história, com vistas a estimá-la, podem vir à tona complexidades e riscos que não haviam sido anteriormente percebidos.

Uma história tem que ser **pequena (*small*)** o suficiente para caber em uma única *sprint*. A mentalidade ágil nos diz que o importante é liberar a história para o usuário em intervalos curtos (*sprints*) para que ela possa ser experimentada pelo usuário, de modo que a equipe de desenvolvimento possa receber rapidamente um *feedback*. Quando uma história é grande demais, ela é considerada um épico e deve ser particionada em histórias menores, que podem ser implementadas em uma *sprint*.

Toda história deve ser **testável (*testable*)**. A ideia é que uma história não deve entrar em uma *sprint* se ela não puder sair e, para sair, ela deve estar testada. Uma história só está pronta para o desenvolvimento se conhecermos os seus critérios de avaliação. Se não soubermos como avaliar uma história, ela não está pronta para ser implementada.

Mapeamento das histórias de usuário

Ao dar início ao mapeamento das histórias de usuário, certamente você vai se deparar com os diversos *stakeholders* que estão envolvidos em um projeto, cada um contribuindo com a sua visão sobre o projeto. Neste momento é possível que você perceba que apenas uma frase, escrita em um cartão ou Post-it®, contendo quem, o que e como não será suficiente.

Patton (2014) usa uma metáfora excelente para essas situações: histórias de usuário, escritas em cartões, são como os antigos cartões de livros das bibliotecas físicas — eles apresentam informações básicas sobre a obra, que permitem que possamos selecioná-la e localizá-la, mas o conteúdo está em outro lugar, ou seja, nos livros.

Haverá casos em que um cartão ou um Post-it® darão conta do recado, mas isto é raro. Na maioria dos casos, precisamos mais do que isso, especialmente porque as regras do negócio raramente são simples.

É bastante comum, atualmente, que as empresas estejam estruturadas no formato de grandes espaços abertos onde suas equipes sentam lado a lado, facilitando a comunicação e a colaboração. Tem sido também comum que as paredes dessas empresas sejam extensivamente utilizadas para expressar histórias, planejamentos, riscos, anotações. Geralmente são paredes de vidro ou paredes revestidas de materiais nos quais se pode escrever e apagar, como grandes quadros brancos. Nesses espaços reside o mapeamento das histórias de usuário. Graças à facilidade da tecnologia, uma foto, ou um vídeo, são suficientes para se guardar seu conteúdo para uso posterior. Esses registros em outros meios, além das notas adesivas e dos rascunhos nas paredes, são importantes para que se resgate o percurso posteriormente, quando necessário.

As duas principais razões pelas quais o mapeamento de histórias funciona, de acordo com Patton (2014), são estas: ele estabelece um entendimento comum por meio de palavras e figuras; e ele força a conversa na direção não apenas do que será construído, mas de quem vai utilizar o que será construído.

De acordo com Patton (2014), os seguintes passos devem ser utilizados para construir a história de usuário:

- **Delimite o problema:** para quem o produto está sendo desenvolvido e por que está sendo desenvolvido.
- **Mapeie a visão geral:** faça um mapa que seja abrangente em largura, mas raso em profundidade. Mapeie o mundo como ele é hoje, com as dores e alegrias do usuário.
- **Explore:** aprofunde, entendendo como as outras pessoas fazem a mesma coisa.
- **Gere uma estratégia de entrega:** sempre existe muita coisa para ser construída, então, priorize o que agrega mais valor.
- **Gere uma estratégia de aprendizagem:** construa o produto mínimo viável e teste com o público-alvo para aprender.
- **Gere uma estratégia de desenvolvimento:** divida o que deverá ser desenvolvido em o que deverá ser desenvolvido agora e depois. Desenvolva o que oferece os maiores riscos primeiro.

3 Aplicação de histórias de usuário

Histórias de usuário diferem de outras abordagens tradicionais de requisitos, de acordo com Leffingwell (2011), em diversos aspectos, acompanhe:

- Elas não são especificações detalhadas de requisitos, mas sim expressões de intenção negociáveis.
- Elas são curtas, fáceis de ler e compreensíveis por *stakeholders*, desenvolvedores e usuários.
- Elas representam pequenos incrementos de funcionalidade de valor, que podem ser implementadas em poucos dias ou semanas.
- Elas são relativamente fáceis de estimar, de modo que o esforço para implementação possa ser rapidamente estimado.
- Elas não são escritas em documentos grandes e pesados, mas organizadas em listas que podem ser arrumadas e rearrumadas mais facilmente à medida que novas informações são descobertas.
- Elas não são detalhadas no início do projeto, mas são elaboradas à medida que são necessárias (*just-in-time*), evitando especificações prematuras, atrasos no desenvolvimento, longo inventário de requisitos e uma declaração da solução muito restritiva.
- Elas requerem pouca ou nenhuma manutenção e podem ser facilmente descartadas depois de implementadas.
- As histórias de usuário, e o código criado rapidamente a partir delas, servem como entradas para documentação, que então é desenvolvida incrementalmente também.

É comum que você escute de uma empresa ágil que utilizar a abordagem de casos de uso reduz a agilidade da empresa, pois traz excesso de burocracia. Na realidade, é preciso compreender que a especificação de casos de uso pode ser descrita na profundidade que a equipe achar necessária para entender claramente o que deve ser construído, nem mais, nem menos. Se os casos de uso forem excessivamente detalhados, e a equipe tem um bom domínio do negócio, então eles se tornarão realmente enfadonhos e é possível que não tenham mesmo utilidade. No entanto, o outro lado também é preocupante, ou seja, ter apenas uma história do usuário registrada em um Post-it® no nível mais alto de um épico não será útil para uma equipe que não conhece a fundo o domínio de negócio.



Saiba mais

Para conhecer mais sobre a escrita efetiva de casos de uso proposta por Alistair Cockburn, consulte o livro *Escrevendo Casos de Uso Eficazes — Um Guia Prático para Desenvolvedores de Software* (COCKBURN, 2005). Nesse livro o autor aborda como escrever casos de uso, nos diversos níveis de abstração, discutindo como aplicar cada elemento desse tipo de especificação de requisitos funcionais. Vale a pena a leitura.

Aplicações, limitações e complementações

Tanto casos de uso quanto histórias do usuário são excelentes para elicitar requisitos de sistemas cujo controle está na mão do usuário, ou seja, sistemas com muitas interações por meio de interfaces com o usuário. No entanto, para os sistemas cuja complexidade não resida na interação propriamente dita, essas duas ferramentas podem não se aplicar tão bem. Esse é o caso de sistemas intensivos em processamento, como rotinas *batch*, processamento de grandes volumes de dados (aplicações de *business intelligence*, *big data*), algoritmos de inteligência artificial etc. Esse também é o caso de sistemas embarcados ou de tempo real (WIEGERS; BEATTY, 2013). Para esses casos, outras abordagens podem ser mais adequadas, como diagramas de máquinas de estado, diagrama de atividades, diagrama de sequência, diagrama de componentes, pseudocódigo etc.

À medida que você for ganhando mais experiência com a especificação de requisitos, vai perceber que pode ainda combinar diversas técnicas e agregar ou eliminar informações na medida da sua necessidade e da equipe de desenvolvimento. Pode ser que, em algum momento, para detalhar um fluxo de um caso de uso ou uma história de usuário, seja necessário, por exemplo, desenhar um diagrama de atividades ou um diagrama de sequência. Isso não é um problema e também não significa que você está deixando de ser ágil porque usou um diagrama UML; significa que você encontrou utilidade em usar as ferramentas de forma complementar.

A mensagem que fica é que o usuário deseja uma funcionalidade que resolva uma dor que ele sente. Para isso, você deve lançar mão da ferramenta da engenharia de requisitos que seja mais adequada àquele contexto, podendo ser um diagrama, um texto, um vídeo ou o que mais a sua imaginação de analista de requisitos criar. Qualquer ferramenta que apoie o diálogo para a compreensão dos requisitos é útil para o desenvolvimento de melhores produtos de *software*.



Referências

BECK, K. *et al. Manifesto para desenvolvimento ágil de software*. [*S. l.*], 2001. Disponível em: https://agilemanifesto.org/iso/ptbr/manifesto.html. Acesso em: 18 mar. 2020.

BERG, V. *et al*. Software startup engineering: a systematic mapping study. *Journal of Systems and Software*, v.144, p. 255–274, 2018.

COCKBURN, A. *Escrevendo casos de uso eficazes*: um guia prático para desenvolvedores de software. Porto Alegre: Bookman, 2005.

COHN, M. *User stories applied for agile software development*. Boston: Pearson Education, 2004.

CROWNE, M. Why Software Product Startups fail and what to do about it. *In*: IEEE INTERNATIONAL ENGINEERING MANAGEMENT CONFERENCE, 2002. *Anais*... Cambridge: IEEE, 2002. p. 338–343.

GIARDINO, C. *et al*. What do we know about software development in startups?. *IEEE Software*, v. 31, n. 5, p. 28–32, 2014.

GIARDINO, C. *et al*. Software Development in Startup Companies: The Greenfield Startup Model. *IEEE Transactions on Software Engineering*, v. 42, n. 6, p. 585–604, 2016.

LEFFINGWELL, D. *Agile software requirements:* lean requirements practices for teams, programs, and the enterprise. Upper Saddle River: Pearson Education, 2011.

NEON RAIN INTERACTIVE. *Agile Scrum for web development*. Denver, 2020. Disponível em: https://www.neonrain.com/agile-Scrum-web-development/. Acesso em: 18 mar. 2020.

PATTON, J. *User story mapping:* discover the whole story, build the right product. Sebastopol: O´Reilly, 2014.

SCHWABER, K.; SUTHERLAND, J. *Guia do SCRUM*: um guia definitivo para o SCRUM: as regras do jogo. [*S. l.*], 2013. Disponível em: https://www.Scrumguides.org/docs/Scrumguide/v1/Scrum-Guide-Portuguese-BR.pdf. Acesso em: 18 mar. 2020.

STELLMAN, A.; GREENE, J. *Head first agile*: a brain-friendly guide. Sebastopol: O´Reilly, 2017.

WIEGERS, K. E.; BEATTY, J. *Software requirements*. 3. ed. Redmond: Microsoft Press, 2013.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

PREZADO ESTUDANTE

ENCERRA AQUI O TRECHO DO LIVRO DISPONIBILIZADO PELA SAGAH PARA ESTA PARTE DA UNIDADE.

CONTRIBUA COM A QUALIDADE DO SEU CURSO

Se você encontrar algum problema neste material, entre em contato pelo email **eadproducao@unilasalle.edu.br.** Descreva o que você encontrou e indique a página.

Lembre-se: a boa educação se faz com a contribuição de todos!

UNIVERSIDADE
LaSalle
Viver é evoluir.

Av. Victor Barreto, 2288 | Canoas - RS
CEP: 92010-000 | 0800 541 8500
eadproducao@unilasalle.edu.br